Background for "What Sheaf Theory Actually Gets You"

PL Foundations • Sheaf Theory • Cohomology

Everything you need to present the deppy_honest_comparison confidently, starting from first principles.

1. **Part I** (Slides 1–60): PL foundations—types, abstract interpretation, refinement types, dependent types, verification conditions
2. **Part II** (Slides 61–120): Sheaf theory—categories, presheaves, sheaves, Grothendieck topologies, sites, descent
3. **Part III** (Slides 121–170): Cohomology—cochain complexes, Čech cohomology, H^0/H^1, Mayer-Vietoris, obstructions
4. **Part IV** (Slides 171–200): Connecting it all—how deppy uses sheaves for bug-finding, equivalence, and spec-checking

Goal: After these slides, every symbol and claim in the 10-slide presentation should feel natural.

Part I

Programming Language Foundations

Types $\rightarrow$ Abstract Interpretation $\rightarrow$ Refinement Types $\rightarrow$
Dependent Types $\rightarrow$ Verification Conditions

Informal: A type classifies values by the operations they support.

Formal (set-theoretic): A type τ denotes a set $[\![\tau]\!]$ of values.

- $[\![\texttt{int}]\!] = \mathbb{Z}$
- $[\![\texttt{bool}]\!] = \{\texttt{True}, \texttt{False}\}$
- $[\![\texttt{int} \to \texttt{int}]\!] = \mathbb{Z} \to \mathbb{Z}$

Why types?

1. *Safety*: prevent nonsensical operations ($\texttt{"hello"} + 3$)
2. *Abstraction*: hide implementation details
3. *Documentation*: machine-checked contracts

A **typing judgment** has the form:

$$\Gamma \vdash e : \tau$$

Read: "In context Γ, expression e has type τ."

Context Γ is a list of variable-type bindings: $x_1 : \tau_1, \ldots, x_n : \tau_n$.

Example rules:

$$\frac{\Gamma \vdash e_1 : \texttt{int} \quad \Gamma \vdash e_2 : \texttt{int}}{\Gamma \vdash e_1 + e_2 : \texttt{int}} \text{ (T-Add)} \qquad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda x.\, e : \tau_1 \to \tau_2} \text{ (T-Abs)}$$

The fundamental theorem of type systems:

Type Soundness (Milner, 1978; Wright & Felleisen, 1994)
Well-typed programs do not go wrong.

Formally: *Progress* + *Preservation*.

Progress If $\vdash e : \tau$, then either e is a value or $e \to e'$.

Preservation If $\vdash e : \tau$ and $e \to e'$, then $\vdash e' : \tau$.

Why this matters for deppy: The sheaf gluing condition is a *generalized* soundness check—local type assignments "glue" iff the program is well-typed.

Logic	**Type Theory**
Proposition P	Type τ
Proof of P	Term $e : \tau$
$P \Rightarrow Q$	Function type $\tau_1 \to \tau_2$
$P \wedge Q$	Product type $\tau_1 \times \tau_2$
$P \vee Q$	Sum type $\tau_1 + \tau_2$
$\forall x.\, P(x)$	Dependent product $\Pi_{x:\tau_1} \tau_2(x)$
$\exists x.\, P(x)$	Dependent sum $\Sigma_{x:\tau_1} \tau_2(x)$

Key insight: Type checking = proof checking. Deppy's "verification conditions" are propositions; solving them is proving them.

Simple types (Java, Go, C):

- `int`, `String`, `List<T>`
- Can catch: wrong argument count, type mismatches
- Cannot catch: division by zero, array out of bounds, null dereference

Richer type systems add expressive power:

1. **Generics/Parametric polymorphism**: $\forall \alpha.\, \alpha \to \alpha$
2. **Refinement types**: $\{x : \texttt{int} \mid x > 0\}$
3. **Dependent types**: $\texttt{Vec}(n) \to \texttt{Vec}(n)$
4. **Linear types**: use each value exactly once

Deppy operates at levels 2–3: refinement + dependent types for Python.

Definition (Cousot & Cousot, 1977)
Abstract interpretation computes an *over-approximation* of all possible runtime states by executing the program on an *abstract domain* instead of concrete values.

Concrete domain: $(\mathcal{P}(\mathbb{Z}), \subseteq)$ — sets of integers.
Abstract domain: e.g., intervals $[a, b]$, signs $\{-, 0, +\}$, octagons.

Galois connection:

$$\alpha : \mathcal{P}(\mathbb{Z}) \rightleftharpoons \widehat{D} : \gamma \qquad \alpha(S) = \text{best abstraction of } S \qquad \gamma(\hat{d}) = \text{concrete values represented by } \hat{d}$$

```python
def f(x: int, y: int):
    z = x + y       # z in [x+y, x+y]
    if z > 0:
        w = 100 / z  # safe: z > 0
    else:
        w = z - 1    # z <= 0, so w <= -1
    return w
```

Interval analysis:

- After branch $z > 0$: $z \in [1, +\infty)$ — division safe
- After branch $z \leq 0$: $z \in (-\infty, 0]$ — no division

The good: Fully automatic, sound, scales to large programs.
The bad: Imprecise (joins lose information), requires widening for loops.

Loops require computing a fixpoint of the abstract transfer function:

$$\hat{d}^{(n+1)} = f^\sharp(\hat{d}^{(n)}) \sqcup \hat{d}^{(n)}$$

Problem: Ascending chains may not stabilize (e.g., $[0,1], [0,2], [0,3], \ldots$).

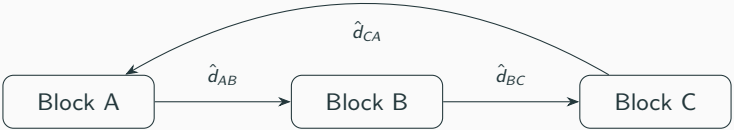
Widening ∇ forces convergence by jumping ahead:

$$\hat{d}^{(n+1)} = \hat{d}^{(n)} \nabla f^\sharp(\hat{d}^{(n)})$$

Cost: Widening loses precision *non-monotonically*—you might overshoot and never recover.

This is exactly what the sheaf approach avoids: checking *agreement* on overlaps doesn't require computing a global fixpoint.

Standard abstract interpretation computes a *single global abstract state*:



With $|D|$ = domain size and k = program variables:

$$\text{Cost} = \mathcal{O}(|D|^k) \quad \text{per fixpoint iteration}$$

Pain points:

- Cost is *exponential* in number of tracked variables
- Widening loses precision globally
- A single imprecise join poisons the entire analysis

Refinement types attach logical predicates to base types:

$$\{x : \tau \mid \varphi(x)\}$$

Examples:

- $\{x : \texttt{int} \mid x > 0\}$ — positive integers
- $\{x : \texttt{int} \mid x \neq 0\}$ — nonzero integers (safe divisors)
- $\{a : \texttt{List}[\texttt{int}] \mid \texttt{len}(a) = n\}$ — length-indexed lists

Subtyping rule:

$$\frac{\forall x.\ \varphi_1(x) \Rightarrow \varphi_2(x)}{\{x : \tau \mid \varphi_1\} <: \{x : \tau \mid \varphi_2\}}$$

Checking subtyping = checking logical implication = **SMT query**.

```haskell
{-@ divide :: Int -> {v:Int | v /= 0} -> Int @-}
divide :: Int -> Int -> Int
divide x y = x `div` y

{-@ average :: {v:[Int] | len v > 0} -> Int @-}
average :: [Int] -> Int
average xs = divide (sum xs) (length xs)
```

Liquid Haskell generates a **verification condition** (VC):

$$\texttt{len}(xs) > 0 \;\Rightarrow\; \texttt{length}(xs) \neq 0$$

and sends it to Z3. If SAT (valid): the program is safe.

Limitation: One monolithic VC per function. Deppy's sheaf approach factors this into per-site VCs.

Dependent types allow types to depend on *values*:

$$\Pi_{n:\mathbb{N}}.\ \mathtt{Vec}(n) \to \mathtt{Vec}(n)$$

"For all n, takes a vector of length n, returns a vector of length n."

Key constructs:

Π-type (dependent function): $\Pi_{x:A} B(x)$ — generalization of $A \to B$

Σ-type (dependent pair): $\Sigma_{x:A} B(x)$ — generalization of $A \times B$

Id-type (identity/equality): $\mathrm{Id}_A(x, y)$ — proof that $x = y$ in A

Refinement types are a *restricted* form: $\{x : \tau \mid \varphi\} \cong \Sigma_{x:\tau} \varphi(x)$ where φ is decidable.

Full dependent types (Coq, Agda, Lean, Idris):

- Extremely expressive: can state any mathematical property
- Type checking is undecidable in general (requires user-provided proofs)
- Manual proof burden is significant

Deppy's sweet spot: Dependent types for Python, but with *automated* proof via SMT.

- User writes: `@deppy.requires(lambda x: x > 0)`
- Deppy infers refinements, generates VCs, calls Z3
- No manual proofs needed for decidable fragments

Deppy's identity type (`IdentityType`): used for equivalence proofs between implementations.

A **verification condition** is a logical formula whose validity implies program correctness.

Standard pipeline:

$$\text{Source code} \xrightarrow{\text{VC gen}} \text{Logical formula} \xrightarrow{\text{SMT solver}} \text{Valid/Invalid} + \text{counterexample}$$

Example: For function with precondition P and postcondition Q:

$$\text{VC} = \forall \vec{x}.\ P(\vec{x}) \ \Rightarrow\ \text{wp}(\text{body}, Q)(\vec{x})$$

where wp is the weakest precondition transformer.

Tools: Dafny, F*, Liquid Haskell, Why3, Frama-C all generate VCs. Deppy does too—but *decomposes* them via the sheaf cover.

Satisfiability Modulo Theories (SMT) solvers decide:

$$\exists \vec{x}.\ \varphi(\vec{x}) \quad \text{over theories } T_1, \ldots, T_k$$

Common theories:

QF_LIA	Quantifier-free linear integer arithmetic (decidable, in NP)
QF_LRA	Quantifier-free linear real arithmetic (decidable, in P)
QF_NIA	Quantifier-free nonlinear integer arithmetic (undecidable!)
QF_BV	Bit-vectors (decidable, NP-complete)
QF_UF	Uninterpreted functions with equality (decidable)

Key for deppy: Each local VC is typically in QF_LIA or QF_UF — cheap to solve. The *locality* of sheaf queries keeps us in decidable fragments.

Problem: Real VCs mix theories (arithmetic + arrays + functions).

Nelson-Oppen (1979): Combine decision procedures for disjoint-signature theories.

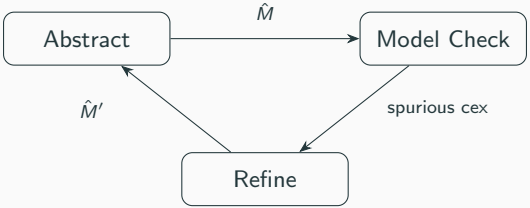
Requirements:

1. Theories share only equality
2. Each theory is *stably infinite* (infinite models exist)
3. Each theory is decidable individually

Algorithm: Propagate equalities between shared variables until fixpoint.

Deppy uses this: The `theory_interface.py` module enforces signature disjointness for theory combination in the solver backend. Tinelli-Zarba (2005) extends Nelson-Oppen to non-stably-infinite theories.

CEGAR (Clarke et al., 2000) iteratively refines an abstraction:



1. Start with coarse abstraction
2. Check property — if holds, done
3. If counterexample is spurious, refine abstraction
4. Repeat

Deppy's CEGAR loop: uses unsat cores from Z3 to identify which predicates to add per iteration.

Control Flow Graph (CFG): Nodes = basic blocks, edges = control flow.

A **basic block** is a maximal sequence of instructions with:

- One entry point (first instruction)
- One exit point (last instruction)
- No branches in the middle

Why CFGs matter for deppy:

- Each basic block (or branch arm) becomes an *observation site*
- Edges between blocks become *overlaps* in the cover
- The CFG structure *is* the topological space on which the sheaf lives

Static Single Assignment (SSA): Every variable is assigned exactly once.

$$\texttt{x = 1; x = x + 1} \quad \rightarrow \quad \texttt{x_1 = 1; x_2 = x_1 + 1}$$

At join points, use ϕ-functions: $\texttt{x_3} = \phi(\texttt{x_1}, \texttt{x_2})$.

SSA in deppy:

- Each SSA value is a potential **site** (kind: `SSA_VALUE`)
- ϕ-nodes are natural **overlap** points between branches
- The `value_lineage.py` module tracks which sites share which SSA values
- The `phi_merge.py` module handles merging sections at join points

Dataflow analysis computes facts that hold at each program point.

Framework: $(L, \sqsubseteq, \sqcup, \sqcap, \bot, \top)$ — a lattice of dataflow facts.

Transfer function: $f_B : L \to L$ for each basic block B. **Join:** $\mathrm{in}(B) = \bigsqcup_{P \in \mathrm{pred}(B)} \mathrm{out}(P)$.

Classic analyses:

- Reaching definitions (which assignments reach which uses)
- Live variables (which variables are read before next write)
- Available expressions (which expressions are already computed)

Connection to deppy: Dataflow = "pushing information forward/backward." Deppy replaces the global fixpoint with *local section inference* + *gluing check*.

Problem: Function calls cross procedure boundaries.

Approaches:

1. **Inlining**: Expand callees. Exponential blowup.
2. **Summaries**: Compute f : input type $\to$ output type once, reuse.
3. **Context sensitivity**: Distinguish call sites (k-CFA, object sensitivity).

Deppy's approach:

- `summary_propagation.py`: Computes function summaries as sections
- `section_transport.py`: Transports sections across call boundaries
- `context_sensitivity.py`: Distinguishes calling contexts
- Result: interprocedural analysis as **section transport on the presheaf**

Modular verification checks each module independently, then composes results.

The compositionality problem:

- Module A passes its own spec
- Module B passes its own spec
- Does $A \circ B$ pass the combined spec?

Standard answer: If interfaces match — yes (*sound but incomplete*).

Sheaf answer: If $H^1 = 0$ — yes (*sound* **and** *complete* for the cover).

This is the key upgrade: Standard modular checking can reject *equivalent* programs with non-matching interfaces. The sheaf approach doesn't.

Question: Do two programs f and g compute the same function?

Why hard:

- Undecidable in general (Rice's theorem)
- Even for terminating programs: exponential state spaces

Approaches:

1. **Bisimulation**: lockstep simulation (exponential)
2. **Translation validation**: check compiler output matches source
3. **Relational verification**: prove $\forall \vec{x}.\ f(\vec{x}) = g(\vec{x})$

Deppy's approach: Decompose the equivalence check into *local* checks on sites, then glue via Čech cohomology. Exponential $\rightarrow$ polynomial when variables partition.

Bisimulation is the gold standard for behavioral equivalence.

Definition: A relation $R \subseteq S_f \times S_g$ is a *bisimulation* if for all $(s, t) \in R$:

1. If $s \xrightarrow{a} s'$, then $\exists t'$ such that $t \xrightarrow{a} t'$ and $(s', t') \in R$.
2. Symmetrically for transitions from t.

Cost: State space is $|S_f| \times |S_g|$. With k variables of domain size d: state space $= d^k \times d^k = d^{2k}$.

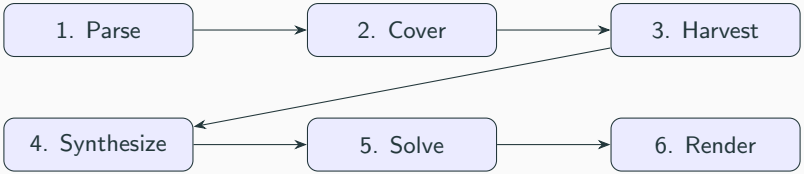
The deppy win: Decompose into m sites each touching k/m variables. Cost: $m \cdot d^{2k/m} \ll d^{2k}$ for $m > 1$.

Industrial analyzers deppy compares against:

Tool	Approach
Astrée	Abstract interpretation (intervals, octagons, polyhedra). Proves *absence* of runtime errors. No user annotations.
Infer	Bi-abduction (separation logic). Memory safety, null pointer, data races.
Pyright/Mypy	Type inference for Python. Structural, no refinement predicates.
Liquid Haskell	Refinement types + SMT. Powerful but requires type annotations.

What none of them provide:

- Compositional obstruction counting ($\mathrm{rk}\, H^1$)
- Behavioral equivalence with completeness guarantees



1. **Parse**: Python AST $\to$ IR (core terms)
2. **Cover**: Build observation sites + overlaps (the topological space)
3. **Harvest**: Infer local sections at each site (the presheaf)
4. **Synthesize**: Forward/backward section propagation (kernel)
5. **Solve**: Check gluing condition, extract H^1 obstructions
6. **Render**: Diagnostics, repair suggestions, SARIF export

#	SiteKind	What it observes
1	`ARGUMENT_BOUNDARY`	Input types at function entry
2	`RETURN_OUTPUT_BOUNDARY`	Output types at function exit
3	`SSA_VALUE`	Type of an SSA-assigned value
4	`BRANCH_GUARD`	Predicate controlling a branch
5	`CALL_RESULT`	Return type of a function call
6	`TENSOR_SHAPE`	Shape constraints on tensors
7	`TENSOR_ORDER`	Dimension count constraints
8	`TENSOR_INDEXING`	Index expression constraints
9	`HEAP_PROTOCOL`	Protocol conformance on heap objects
10	`PROOF`	User-supplied proof terms
11	`TRACE`	Dynamic trace observations
12	`ERROR`	Error/exception sites
13	`LOOP_HEADER_INVARIANT`	Invariant at loop headers
14	`MODULE_SUMMARY`	Cross-module type summaries

Not all type information is equally reliable. Deppy tracks **trust**:

TrustLevel	**Meaning**
`PROOF_BACKED`	Backed by SMT proof or user-supplied proof term
`TRUSTED_AUTO`	Inferred by static analysis with high confidence
`BOUNDED_AUTO`	Bounded inference (e.g., k unrollings)
`TRACE_BACKED`	Supported by dynamic traces (testing)
`RESIDUAL`	Default / not yet classified
`ASSUMED`	User-asserted without proof

Trust levels form a *partial order*. Higher trust = stronger evidence. The gluing check accounts for trust: disagreements involving `PROOF_BACKED` sections are more severe.

```python
@dataclass
class LocalSection:
    site_id: SiteId                     # Which site this section lives at
    carrier_type: Any = None            # The base type (e.g., int, str)
    refinements: Dict[str, Any] = {}    # Refinement predicates
    trust: TrustLevel = RESIDUAL        # How reliable is this info?
    provenance: Optional[str] = None    # Where did this come from?
    witnesses: Dict[str, Any] = {}      # Proof terms / evidence
```

Analogy: A local section is like a local coordinate chart in differential geometry. It describes the "type landscape" at one site.

Refinements dict: maps variable names to predicates. E.g., {`"x": "x > 0", "y": "isinstance(y, int)"`}.

Overlap: Two sites U_i, U_j overlap when they share variables or control flow.

Gluing condition: The local sections must *agree* on the shared part:

$$s_i|_{U_i \cap U_j} = s_j|_{U_i \cap U_j}$$

In deppy terms:

- s_i = section at site i (e.g., "`x` has type `int`, `x > 0`")
- s_j = section at site j (e.g., "`x` has type `int`, `x` is even")
- Restriction to overlap: look at shared refinement keys
- Agreement: the predicates on shared keys must be compatible

When they disagree: That's a *type error* — an obstruction in H^1.

Abstract Interpretation	**Sheaf Approach**
Compute global fixpoint	Infer local sections independently
Cost $\mathcal{O}(\lvert D\rvert^k)$	Cost $\mathcal{O}(n)$ local + $\mathcal{O}(\lvert\text{overlaps}\rvert)$ gluing
Widening loses precision	No widening needed
Single imprecise join poisons all	Imprecision stays local
Loop invariants required	Check agreement, not invariants
Monolithic: all or nothing	Incremental: change 1 site, re-check neighbors

Key insight from the honest comparison:
Checking $\rho_{U_i \cap U_j}$ (restriction to overlap) is total
$\Rightarrow$ checking *agreement* is cheaper than constructing a global section.

Gap predicate: The formula witnessing a gluing failure:

$$\text{gap} = \text{avail}_i \wedge \neg\, \text{req}_j$$

avail_i = what site i provides (e.g., "$\mathtt{x}$ can be 0")
req_j = what site j requires (e.g., "$\mathtt{x} \neq 0$")

Bug detection:

1. Compute gap predicate for each overlap
2. Ask Z3: Is gap satisfiable?
3. If SAT: the satisfying assignment is a *concrete bug witness*
4. If UNSAT: this overlap is safe

Each query is **local** (involves only variables at two adjacent sites), so it's fast and stays in decidable SMT fragments.

Obstruction: gap predicate $\varphi = (y = 0)$ at some overlap.

Repair: negate the gap to get a guard:

$$\text{repair_guard} = \neg\varphi = (y \neq 0)$$

Translation to code:

- Gap: "y can be 0 here"
- Repair: "add `if y != 0:` before the division"

Why this is better than "division by zero on line 42":

1. The gap predicate is *precise*: it says exactly what's wrong
2. The repair guard is *constructive*: it says exactly what to add
3. Multiple gaps may share structure: the H^1 basis tells you the *minimum* number of independent fixes

Problem: Predicates can be syntactically different but semantically equal.

Knuth-Bendix completion: Transform equations into a confluent rewrite system.

1. Orient equations using a term ordering (RPO)
2. Compute critical pairs (overlapping rewrites)
3. Add new rules to resolve critical pairs
4. Repeat until confluent (or fail)

In deppy: The `knuth_bendix.py` module normalizes predicates so that

- `x + y > 0` and `y + x > 0` produce the same normal form (AC-normalization)
- `x > 0 && x > 0` simplifies to `x > 0`
- Z3 simplify → term convert → AC-normalize → KB-normalize → back to Z3

Not all constraints are equally hard. Deppy classifies each VC:

Fragment	Theory	Complexity
Linear shape	QF_LIA	P
Broadcast	element-wise max	P
Matrix multiply	inner-dim equality	P
Reshape	product-equality	NP-hard
Device	finite domain	$\mathcal{O}(1)$
Phase	finite domain	$\mathcal{O}(1)$

Why this matters: By decomposing via the sheaf cover, each local VC tends to land in a *cheaper* fragment than the monolithic VC would.

A proof certificate is machine-checkable evidence that a VC holds.

Deppy's certificate strategies (in order of preference):

1. **Z3 native proof**: Extract Z3's internal proof object
2. **UNSAT core**: Minimal set of assertions implying UNSAT
3. **Replay**: Re-check with fresh solver
4. **Dual solver**: Cross-check with CVC5

Each step in a proof certificate:

- `rule`: the inference rule (modus ponens, theory lemma, rewrite, ...)
- `conclusion`: the SMT-LIB formula proved
- `premises`: indices of earlier steps used

Open goal: export to Lean/Coq for formal verification.

What you need to remember for the presentation:

1. **Types** classify values; **type soundness** = well-typed programs don't crash
2. **Abstract interpretation** computes over-approximations via global fixpoints — *expensive, imprecise for loops*
3. **Refinement types** attach predicates ($\{x : \tau \mid \varphi\}$); checking = SMT
4. **Dependent types** let types depend on values (Π, Σ, Id)
5. **Verification conditions** are the formulas to prove; standard tools generate one big VC per function
6. **SMT solvers** decide these formulas; *locality* keeps us in decidable fragments
7. **The compositionality gap**: standard tools check modules independently (sound but incomplete); sheaf theory fills this gap

Definition: A *partial order* $(P, \sqsubseteq)$ is a set with a reflexive, antisymmetric, transitive relation.

A *lattice* adds:

- **Join** $a \sqcup b$: least upper bound
- **Meet** $a \sqcap b$: greatest lower bound
- $\bot$ (bottom): least element
- $\top$ (top): greatest element

In abstract interpretation: The abstract domain is a lattice. **In sheaf theory:** The stalk at each point is partially ordered (information order). **In deppy:** `InformationOrder` protocol — sections at each site are ordered by how much they know.

Definition: A *Galois connection* between posets $(C, \leq)$ and $(A, \sqsubseteq)$ is a pair (α, γ):

$$\alpha(c) \sqsubseteq a \quad \Leftrightarrow \quad c \leq \gamma(a)$$

Properties:

- α is the *abstraction* function (concrete $\to$ abstract)
- γ is the *concretization* function (abstract $\to$ concrete)
- $\alpha \circ \gamma \circ \alpha = \alpha$ (round-tripping)
- $\gamma \circ \alpha \geq \mathrm{id}_C$ (abstraction over-approximates)

Connection to sheaves: A restriction map in a presheaf is like α—going from a larger site to a smaller one loses information, just like abstracting.

Domain	Tracks	Cost
Signs $\{-, 0, +, \top\}$	Sign of variables	$\mathcal{O}(n)$
Intervals $[a, b]$	Range of variables	$\mathcal{O}(n)$
Octagons $\pm x \pm y \leq c$	Pairwise relations	$\mathcal{O}(n^3)$
Polyhedra $Ax \leq b$	All linear relations	$\mathcal{O}(2^n)$

Precision–cost tradeoff: More precise domains $\Rightarrow$ exponentially more expensive.

Deppy's insight: Instead of choosing one domain for the whole program, use *different* local inference methods at different sites (AI, symbolic exec, ML, tests). The sheaf framework composes them.

Predicate abstraction (Graf & Saidi, 1997): Track truth values of a fixed set of predicates.

Given predicates $p_1, \ldots, p_k$, abstract state = element of $\{0,1\}^k$.

Example: Predicates $\{x > 0, y \neq 0, x < y\}$

- State $(1,1,0)$: "$x > 0$, $y \neq 0$, $x \geq y$"
- Precise for these properties, oblivious to others

CEGAR + predicate abstraction:

1. Start with few predicates
2. Find spurious counterexample
3. Add new predicates to eliminate it
4. Repeat (converges for finite-state properties)

Deppy's CEGAR loop works similarly, but on a per-site basis.

Separation logic (Reynolds, 2002): Reason about heap-manipulating programs.

Key connective: $P * Q$ — "P and Q hold on *disjoint* heap regions."

Frame rule:

$$\frac{\{P\}\ c\ \{Q\}}{\{P * R\}\ c\ \{Q * R\}}$$

"If c only touches the P part, the R part is untouched."

Connection to sheaves:

- Separation = disjoint sites
- Frame rule = local reasoning (modify one site, others unchanged)
- Sheaf gluing = composing local heap assertions

Python, JavaScript, Ruby: Types determined at runtime.

Challenges for static analysis:

- Duck typing: `x.foo()` — what is `x`?
- Dynamic dispatch: method determined at runtime
- Metaprogramming: `getattr(obj, name)`
- Eval: `eval("x + " + str(y))`

Deppy's approach:

1. Parse Python AST (handle generators, decorators, comprehensions)
2. Infer type guards from control flow (`isinstance`, `is None`)
3. Use dynamic traces as `TRACE_BACKED` sections
4. Let the sheaf framework compose static + dynamic evidence

Python narrows types via control flow:

```python
def safe_div(x: int, y: int) -> float:
    if y != 0:            # Branch guard: y != 0
        return x / y      # Safe: y != 0 in this branch
    return 0.0
```

In deppy:

- `BRANCH_GUARD` site at the `if`: predicate $y \neq 0$
- True-branch `SSA_VALUE` site: inherits $y \neq 0$
- False-branch: inherits $y = 0$
- Overlap between guard and branch: the restriction map carries the guard predicate

The `guard_extractor.py` module extracts these predicates from AST patterns.

Forward analysis: Push information from inputs to outputs.

$$\text{section}(\text{output}) = \text{transfer}(\text{section}(\text{input}))$$

Backward analysis: Pull requirements from outputs to inputs.

$$\text{section}(\text{input}) = \text{wp}(\text{body}, \text{section}(\text{output}))$$

Deppy's kernel (`forward_synthesis.py` + `backward_synthesis.py`):

- Forward: propagate known types from arguments inward
- Backward: propagate required types from return sites outward
- Meet in the middle: check gluing at overlaps

This is **bidirectional type checking** lifted to the presheaf level.

Deppy's `fixed_point.py`: Iterate forward + backward until sections stabilize.

1. Initialize all local sections to $\perp$ (no information)
2. Forward pass: propagate from inputs
3. Backward pass: propagate from outputs
4. Check: did any section change?
5. If yes: repeat. If no: done.

Key difference from classical abstract interpretation:

- Classical: global fixpoint on one big domain
- Deppy: fixpoint on *sections of a presheaf* — conceptually similar but structured by the topology
- After fixpoint: check *gluing* (agreement on overlaps)
- Gluing failures = type errors, *not* imprecision artifacts

At CFG join points, two branches contribute different sections.

Classical: $\phi(x_1, x_2)$ merges values. Abstract interpretation: $\text{join}(\hat{d}_1, \hat{d}_2)$.

Deppy's `phi_merge.py`**:**

- Don't join eagerly — keep both sections
- Record the overlap between the two branches
- Let the gluing check handle consistency
- Only merge when both sections agree (or when forced by downstream uses)

Benefit: No precision loss from premature joins. If the branches provide contradictory information, that shows up as an H^1 obstruction, not a silently widened abstract value.

Deppy detects **20 classes** of bugs, zero annotations:

1. Division by zero
2. Null/None dereference
3. Type mismatch (int/str/...)
4. Index out of bounds
5. Key error
6. Attribute error
7. Uninitialized variable
8. Dead code
9. Unreachable code
10. Missing return
11. Redundant condition
12. Integer overflow
13. Format string error
14. Assertion failure
15. Resource leak
16. Data race
17. Use after free
18. Double free
19. Buffer overflow
20. Protocol violation

Task 2 from the honest comparison: behavioral equivalence.

Input: Two functions f and g. **Output:** Equivalent (with proof) or not (with localized counterexample).

Pipeline:

1. Build covers for f and g (same site structure)
2. Compute local equivalence judgments at each site
3. Build transition functions on overlaps (the 1-cocycle)
4. Compute H^1 via Čech cohomology
5. $H^1 = 0 \Rightarrow$ global equivalence; $H^1 \neq 0 \Rightarrow$ localized failure

The equivalence module: `deppy/equivalence/` — cohomology.py, descent.py, global_checker.py, local_checker.py, obstruction.py.

Task 3: Refinement-type spec vs. function $\rightarrow$ yes/no.

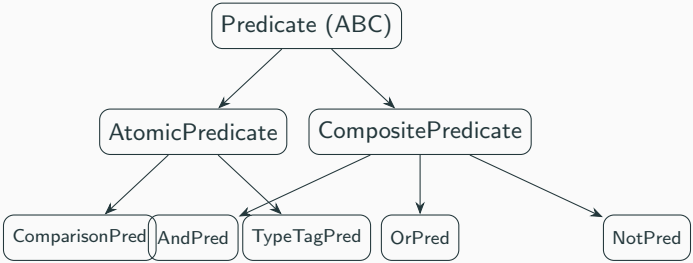
Standard approach: Generate one big VC, send to solver.

Deppy's approach: Product cover factoring.

- Cover $= \{(\text{path}_i, \text{conjunct}_j) \mid i \in \text{paths}, j \in \text{spec}\}$
- Each VC involves one path and one spec conjunct — *small*
- All $n \times k$ VCs are independent $\Rightarrow$ parallelizable
- Change one path: re-check only that row
- Failure localized: "conjunct 2 fails on path 4"

Meta-soundness: $H^0 = 1$ (cover connected) $\wedge$ $H^1 = 0$ (proofs consistent) $\wedge$ all VCs pass $\Rightarrow$ spec holds with structured certificate.

Deppy's type system is built on a **predicate algebra**:



Every predicate supports: `free_vars()`, `substitute_var()`, `negate()`.

These are the "values" that sections carry. Agreement between sections is checked by comparing their predicates via SMT.

Formally: Deppy works with a *category* **S** of observation sites.

Objects	Sites (the 14 families)
Morphisms	Restriction maps (inclusion of one site's scope into another's)
Composition	Restriction is transitive
Identity	Every site restricts to itself

The **semantic presheaf** is a functor $\mathrm{Sem} : \mathbf{S}^{\mathrm{op}} \to \mathbf{Poset}$:

- To each site, assign its set of possible sections (ordered by information)
- To each morphism, assign a restriction function

This is where category theory enters. Let's build up to it properly.

1. **Slide 3** mentions abstract interpretation + widening + precision loss.
   $\to$ You now know why: global fixpoint, $\mathcal{O}(|D|^k)$, widening overshoots.
2. **Slide 4** claims $\operatorname{rk} H^1$ = minimum independent fixes.
   $\to$ You need sheaf theory for this. Coming in Part II–III.
3. **Slide 5** claims exponential savings via decomposition.
   $\to$ $m \cdot d^{k/m} \ll d^k$. The "sites" are code regions.
4. **Slide 7** mentions VCs and product covers.
   $\to$ Standard VC gen gives one big formula. Deppy factors it.
5. **Slide 8** mentions H^0, H^1 certificates.
   $\to$ You need cohomology. Coming in Part III.
6. **Slide 9** uses the Mayer-Vietoris exact sequence.
   $\to$ The crown jewel. Explained in Part III.

Term	One-line definition
Abstract interpretation	Over-approximate program behavior on an abstract domain
Galois connection	Soundness link between concrete and abstract domains
Widening	Force convergence of ascending chains (loses precision)
Refinement type	Base type + logical predicate: $\{x : \tau \mid \varphi\}$
Dependent type	Type that depends on a value: $\Pi_{n:\mathbb{N}} \mathtt{Vec}(n)$
Verification condition	Logical formula whose validity $\Rightarrow$ program correctness
SMT solver	Decision procedure for satisfiability in combined theories
CEGAR	Iterative refinement of abstractions using counterexamples
Bisimulation	Stepwise behavioral equivalence relation
SSA	Static single assignment: each variable assigned once

Term	One-line definition
Knuth-Bendix	Complete equational theories into confluent rewrite systems
Nelson-Oppen	Combine decision procedures for disjoint-signature theories
Separation logic	Reason about disjoint heap regions via $*$ connective
Frame rule	Local changes don't affect disjoint state
Type soundness	Well-typed programs don't get stuck
Curry-Howard	Proofs = programs, propositions = types
Predicate abstraction	Track boolean vector of predicate truth values
Fixpoint	Invariant solution of transfer-function iteration
Weakest precondition	Most permissive precondition ensuring a postcondition
Proof certificate	Machine-checkable evidence of validity

$\Gamma \vdash e : \tau$	Expression e has type τ in context Γ
$\{x : \tau \mid \varphi\}$	Refinement type: τ constrained by φ
$\Pi_{x:A} B(x)$	Dependent function type
$\Sigma_{x:A} B(x)$	Dependent pair type
$\mathrm{Id}_A(a, b)$	Identity/equality type in A
α, γ	Abstraction and concretization (Galois connection)
∇	Widening operator
$\mathrm{wp}(c, Q)$	Weakest precondition of c for postcondition Q
$\sqsubseteq, \sqcup, \sqcap$	Partial order, join, meet in a lattice

Part II

Sheaf Theory

Categories $\to$ Presheaves $\to$ Sheaves $\to$
Grothendieck Topologies $\to$ Descent

Definition
A *category* $\mathbf{C}$ consists of:

- A collection of *objects* $\mathrm{Ob}(\mathbf{C})$
- For each pair of objects A, B: a set of *morphisms* $\mathrm{Hom}(A, B)$
- Composition: $g \circ f : A \to C$ whenever $f : A \to B$ and $g : B \to C$
- Identity: $\mathrm{id}_A : A \to A$ for each object A

Laws:

1. Associativity: $(h \circ g) \circ f = h \circ (g \circ f)$
2. Identity: $f \circ \mathrm{id}_A = f = \mathrm{id}_B \circ f$

Category	Objects	Morphisms
Set	Sets	Functions
Ab	Abelian groups	Group homomorphisms
Top	Topological spaces	Continuous maps
$\mathbf{Vect}_k$	Vector spaces over k	Linear maps
Poset	Posets	Order-preserving maps
$\mathrm{Open}(X)$	Open sets of X	Inclusions $U \hookrightarrow V$

For deppy: $\mathbf{S}$ = the *site category*, with:

- Objects = observation sites (14 families)
- Morphisms = restriction/reindexing maps

Definition
A *functor* $F : \mathbf{C} \to \mathbf{D}$ maps:

- Objects: $A \mapsto F(A)$
- Morphisms: $(f : A \to B) \mapsto (F(f) : F(A) \to F(B))$

preserving composition and identities: $F(g \circ f) = F(g) \circ F(f), \quad F(\mathrm{id}_A) = \mathrm{id}_{F(A)}$.

A presheaf is a functor $\mathcal{F} : \mathbf{C}^{\mathrm{op}} \to \mathbf{Set}$ (or $\to \mathbf{Poset}$).

The "op" reverses arrows: if $U \hookrightarrow V$ in the site category, the presheaf gives a *restriction* map $\mathcal{F}(V) \to \mathcal{F}(U)$ going the other way.

Definition: $\mathbf{C}^{\mathrm{op}}$ has the same objects as $\mathbf{C}$, but all morphisms reversed:

$$\mathrm{Hom}_{\mathbf{C}^{\mathrm{op}}}(A, B) = \mathrm{Hom}_{\mathbf{C}}(B, A)$$

Why? A presheaf $\mathcal{F} : \mathbf{C}^{\mathrm{op}} \to \mathbf{Set}$ is a *contravariant* functor on $\mathbf{C}$.

Intuition:

- In $\mathrm{Open}(X)$: $U \subseteq V$ gives inclusion $U \hookrightarrow V$
- A presheaf reverses this: $\mathcal{F}(V) \to \mathcal{F}(U)$ (restrict from bigger to smaller)
- Going to a *smaller* open set $=$ looking *more locally*

Definition
A *natural transformation* $\eta : F \Rightarrow G$ between functors $F, G : \mathbf{C} \to \mathbf{D}$ assigns to each object A a morphism $\eta_A : F(A) \to G(A)$ such that for every $f : A \to B$:

$$\eta_B \circ F(f) = G(f) \circ \eta_A$$

$$\begin{tikzcd}
F(A) \arrow[r, "\eta_A"] \arrow[d, "F(f)"] & G(A) \arrow[d, "G(f)"] \\
F(B) \arrow[r, "\eta_B"] & G(B)
\end{tikzcd}$$

For deppy: A natural transformation between presheaves = a way to compare the "type information" assigned by two different analyses, site by site, consistently.

Definition
A *topological space* (X, τ) is a set X with a collection τ of "open sets" such that:

1. $\emptyset, X \in \tau$
2. Arbitrary unions of open sets are open
3. Finite intersections of open sets are open

Why topology? Sheaf theory was born here.

- X = a geometric space (manifold, variety, scheme)
- Open sets = regions where we can make local observations
- Restriction: zoom in from a big region to a smaller one

For deppy: X = the program. Open sets = code regions (sites). Topology = which regions overlap.

Definition
An *open cover* of $U \subseteq X$ is a collection $\{U_i\}_{i \in I}$ of open sets such that $U = \bigcup_{i \in I} U_i$.

Example: Cover the circle S^1 with two arcs:

- U_1 = top half (plus overlap)
- U_2 = bottom half (plus overlap)
- $U_1 \cap U_2$ = two small arcs (left and right overlaps)

For deppy:

- Cover = `Cover` object: `sites`, `morphisms`, `overlaps`
- Sites = code regions; overlaps = shared variables / control flow
- `cover_builder.py` constructs covers from the CFG

Definition
A *presheaf* on a topological space X is a functor $\mathcal{F} : \mathrm{Open}(X)^{\mathrm{op}} \to \mathbf{Set}$.

Concretely: for each open U, a set $\mathcal{F}(U)$ ("sections over U"), and for each inclusion $U \subseteq V$, a *restriction map* $\rho_{V,U} : \mathcal{F}(V) \to \mathcal{F}(U)$.

Axioms:

1. $\rho_{U,U} = \mathrm{id}$ (restricting to yourself does nothing)
2. $\rho_{V,U} \circ \rho_{W,V} = \rho_{W,U}$ (restriction is transitive)

Example: $\mathcal{F}(U) = \{\text{continuous functions } U \to \mathbb{R}\}$. Restriction = literal restriction of a function to a smaller domain.

1. **Continuous functions**: $\mathcal{F}(U) = C^0(U, \mathbb{R})$. Restriction: $f|_V$ for $V \subseteq U$.
2. **Smooth functions**: $\mathcal{F}(U) = C^\infty(U, \mathbb{R})$.
3. **Holomorphic functions**: $\mathcal{F}(U) = \mathcal{O}(U)$ on a complex manifold.
4. **Constant presheaf**: $\mathcal{F}(U) = A$ for all U. Restriction $= \mathrm{id}$.
5. **Locally constant functions**: $\mathcal{F}(U) = \{f : U \to A \mid f \text{ locally constant}\}$.

Deppy's presheaf: $\mathrm{Sem}(U_i) = \{\text{possible type sections at site } U_i\}$, ordered by information content. Restriction $=$ forgetting information about variables not in scope at the smaller site.

A section $s \in \mathcal{F}(U)$ is an element of the presheaf at open set U.

Global section: $s \in \mathcal{F}(X)$ — defined on the entire space. **Local section:** $s \in \mathcal{F}(U)$ — defined on one open set.

Notation: $\Gamma(U, \mathcal{F}) = \mathcal{F}(U)$.

$\Gamma(X, \mathcal{F})$ = space of global sections.

In deppy:

- `LocalSection` = section at one site
- `GlobalSection` = collection of compatible local sections
- The whole point: can we *assemble* local sections into a global one?

A presheaf $\mathcal{F}$ is a **sheaf** if for every open cover $\{U_i\}$ of U:

Locality (Separation)
If $s, t \in \mathcal{F}(U)$ satisfy $s|_{U_i} = t|_{U_i}$ for all i, then $s = t$.

Gluing
If sections $s_i \in \mathcal{F}(U_i)$ satisfy $s_i|_{U_i \cap U_j} = s_j|_{U_i \cap U_j}$ for all i, j, then there exists $s \in \mathcal{F}(U)$ with $s|_{U_i} = s_i$ for all i.

In one sentence: Compatible local data can be uniquely assembled into global data.

In deppy terms: If all sites' type sections agree on overlaps, there's a unique consistent global typing.

A presheaf that is NOT a sheaf:

$\mathcal{F}(U) = \{\text{bounded functions } U \to \mathbb{R}\}$.

Take $U_1 = (0,1)$, $U_2 = (1,2)$, $U_3 = (2,3)$, ...

Each f_n bounded on U_n, but $\bigcup f_n$ may be unbounded on $\bigcup U_n$.

The local sections "glue" set-theoretically, but the global function isn't in $\mathcal{F}(\bigcup U_n)$.

Why this matters for PL:

- Local type assignments might be individually consistent...
- ...but fail to glue globally (e.g., contradictory types at overlaps)
- The failure to glue is precisely an **obstruction**: a nontrivial H^1 class

The sheaf condition can be stated as an *exact sequence*:

$$0 \to \mathcal{F}(U) \xrightarrow{\text{res}} \prod_i \mathcal{F}(U_i) \overset{p}{\underset{q}{\rightrightarrows}} \prod_{i<j} \mathcal{F}(U_i \cap U_j)$$

where p and q restrict from different sides of the overlap.

Meaning:

- $\mathcal{F}(U)$ is the *equalizer* of p and q
- A global section = a tuple of local sections where both restriction paths agree
- This is the "kernel" of the difference map $p - q$

This exact sequence is the beginning of the Čech complex (Part III).

Definition
The *stalk* of $\mathcal{F}$ at a point $x \in X$ is:

$$\mathcal{F}_x = \varinjlim_{U \ni x} \mathcal{F}(U) = \{(U, s) \mid x \in U, s \in \mathcal{F}(U)\}/\sim$$

where $(U, s) \sim (V, t)$ if $s|_W = t|_W$ for some $W \subseteq U \cap V$ with $x \in W$.

Intuition: The stalk is the "germ" of all sections at x. Two sections that agree near x are identified.

In deppy: `stalk.py` computes stalks for the equivalence checker. The stalk at a program point = the "ultimate local type" considering all surrounding contexts.

Key property: A morphism of sheaves is an isomorphism iff it's an isomorphism on all stalks.

```python
@dataclass(frozen=True)
class ConcreteMorphism:
    _source: SiteId    # larger site
    _target: SiteId    # smaller site
    projection: Optional[Dict[str, str]] = None

    def restrict(self, section: LocalSection) -> LocalSection:
        """Restrict section from source to target."""
        # Keep only the refinements relevant to the target
        if self.projection:
            new_refs = {k: section.refinements[v]
                        for k, v in self.projection.items()
                        if v in section.refinements}
        else:
            new_refs = dict(section.refinements)
        return LocalSection(
            site_id=self._target,
            carrier_type=section.carrier_type,
            refinements=new_refs,
            trust=section.trust,
        )
```

Restriction = project out irrelevant variables. Exactly like the math.

Not every presheaf is a sheaf. But there's a universal fix:

Sheafification
For any presheaf $\mathcal{F}$, there exists a sheaf $\mathcal{F}^+$ and a map $\theta : \mathcal{F} \to \mathcal{F}^+$ such that:

1. θ preserves stalks: $\mathcal{F}_x \cong \mathcal{F}^+_x$ for all x
2. θ is universal: any map from $\mathcal{F}$ to a sheaf factors through $\mathcal{F}^+$

Construction: Two rounds of the "étalé space" construction (plus construction).

For deppy: The initial local inference gives a presheaf. The gluing check + H^1 computation is essentially asking: *how far is this presheaf from being a sheaf?* The obstructions measure the failure.

Definition
A *morphism* of sheaves $\varphi : \mathcal{F} \to \mathcal{G}$ is a natural transformation: for each U, a map $\varphi_U : \mathcal{F}(U) \to \mathcal{G}(U)$ compatible with restrictions.

In deppy: `sheaf_morphism.py` defines maps between presheaves.

Used for:

- **Equivalence checking**: Is there an isomorphism $\mathrm{Sem}_f \cong \mathrm{Sem}_g$?
- **Refinement checking**: Is there a monomorphism Spec $\hookrightarrow \mathrm{Sem}_f$?
- **Proof transport**: Transporting proofs from one site to another

Key: Isomorphisms can be checked stalk-by-stalk (a local property).

Alternative view of a sheaf: Instead of assigning sets to opens, build a space:

$$\text{Ét}(\mathcal{F}) = \coprod_{x \in X} \mathcal{F}_x \quad \text{(disjoint union of all stalks)}$$

with a natural projection $\pi : \text{Ét}(\mathcal{F}) \to X$.

Sections of the étalé space = sections of the sheaf.

Intuition: Picture a "field" over X where at each point there's a fiber (the stalk). A section is a continuous choice of one element from each fiber.

For deppy: Think of each program point having a "fiber" of possible type assignments. A global section picks one assignment per point, consistently.

Problem: Not everything has a classical topology (e.g., algebraic geometry, PL).

Solution: Grothendieck generalized "open cover" to work on *any* category.

Grothendieck Topology (simplified)
A *Grothendieck topology* on a category $\mathbf{C}$ assigns to each object U a collection of *covering families* $\{U_i \to U\}$ satisfying:

1. **Identity**: $\{\mathrm{id} : U \to U\}$ is a cover
2. **Stability**: Covers pull back: if $\{U_i \to U\}$ covers and $V \to U$, then $\{U_i \times_U V \to V\}$ covers
3. **Transitivity**: Covers of covers are covers

Definition
A *site* is a pair $(\mathbf{C}, J)$ where $\mathbf{C}$ is a category and J is a Grothendieck topology on $\mathbf{C}$.

A **sheaf on a site** $(\mathbf{C}, J)$ is a presheaf $\mathcal{F} : \mathbf{C}^{\mathrm{op}} \to \mathbf{Set}$ satisfying the sheaf condition for every J-cover.

In deppy:

- $\mathbf{C} = \mathbf{S}$ (site category of 14 families)
- $J =$ determined by the cover builder (which overlaps are "real")
- The `GrothendieckTopology` protocol enforces the three axioms
- Sheaf condition checked by `section_gluing.py`

This is the most general setting. It works for programs because programs don't have a classical topology—but they do have a site category.

Classical topology fails for programs:

- No meaningful metric on code
- "Open sets" of program points aren't literally open
- Overlaps are syntactic (shared variables), not geometric

Grothendieck topology works:

- Objects = code regions (any granularity)
- Morphisms = inclusions (this region is part of that one)
- Covers = "enough regions to see everything"
- The axioms are about *composability*, not geometry

The observation site category **S** with its covering families is a genuine site in the Grothendieck sense. This is what makes the cohomological machinery applicable.

For a topological space X:

- **Category**: $\mathrm{Open}(X)$ — objects are open sets, morphisms are inclusions
- **Topology**: A family $\{U_i \hookrightarrow U\}$ covers iff $\bigcup U_i = U$
- **Sheaf**: The usual notion from topology

Key properties:

- Every inclusion $U_i \hookrightarrow U$ is a monomorphism
- Fiber products exist: $U_i \times_U U_j = U_i \cap U_j$
- The nerve of the cover gives the Čech complex

Deppy's site category has all these properties (by construction).

Definition
The *fiber product* $A \times_C B$ is the universal object making the square commute:

$$\begin{tikzcd} A \times_C B \arrow[r] \arrow[d] & B \arrow[d, "g"] \\ A \arrow[r, "f"] & C \end{tikzcd}$$

In $\mathrm{Open}(X)$**:** $U_i \times_U U_j = U_i \cap U_j$ (intersection).

In deppy: `fiber_product.py` computes the "overlap" between two sites:

- Shared variables
- Shared control-flow paths
- The intersection of their scopes

This is used in the equivalence checker to compute transition functions on overlaps.

Definition
Descent data for a cover $\{U_i \to U\}$ of an object in a sheaf consists of:

1. **Local objects**: sections $s_i \in \mathcal{F}(U_i)$
2. **Transition maps**: isomorphisms $\varphi_{ij} : s_i|_{U_{ij}} \xrightarrow{\sim} s_j|_{U_{ij}}$
3. **Cocycle condition**: $\varphi_{jk} \circ \varphi_{ij} = \varphi_{ik}$ on triple overlaps

Descent holds if every descent datum comes from a global object.

In deppy:

- Local objects = local sections at each site
- Transition maps = how sections at adjacent sites relate
- Cocycle condition = checked by `cohomology.py`
- Descent holds iff $H^1 = 0$

Descent holds (= the cover is an "effective epimorphism") when:

$$\mathcal{F}(U) \xrightarrow{\sim} \mathrm{DescentData}(\{U_i \to U\}, \mathcal{F})$$

Meaning: Specifying a global section is *equivalent* to specifying compatible local sections.

When descent fails:

- The local sections don't extend to a global one
- The obstruction lives in H^1 (first cohomology)
- Deppy reports this as: "these sites have incompatible type assignments"

For PL: Descent failure = type error. Descent success = well-typed program (relative to the cover).

Definition
The *nerve* $N(\mathcal{U})$ of a cover $\mathcal{U} = \{U_i\}$ is a simplicial complex:

- 0-simplices: U_i (the sites)
- 1-simplices: $U_i \cap U_j \neq \emptyset$ (pairwise overlaps)
- 2-simplices: $U_i \cap U_j \cap U_k \neq \emptyset$ (triple overlaps)
- $\vdots$

The nerve captures the combinatorial structure of the cover.

In deppy: `Cover.overlaps` = the 1-skeleton of the nerve. Triple overlaps are computed on demand by the cohomology module.

The Čech cohomology is computed from this nerve.

```python
def f(x: int, y: int) -> int:
    if x > 0:          # site U1 (branch guard)
        z = x + y      # site U2 (SSA value)
    else:
        z = x - y      # site U3 (SSA value)
    return z / y       # site U4 (error site: y=0?)
```

Cover:

- U_1: branch guard ($x > 0$)
- U_2: true branch ($z = x + y$, inherits $x > 0$)
- U_3: false branch ($z = x - y$, inherits $x \leq 0$)
- U_4: return site (z/y)
- Overlaps: (U_1, U_2), (U_1, U_3), (U_2, U_4), (U_3, U_4)

Obstruction: At overlap (U_2, U_4) and (U_3, U_4): site U_4 requires $y \neq 0$, but neither U_2 nor U_3 guarantees it.

Deppy's presheaf:

$$\mathrm{Sem} : \mathbf{S}^{\mathrm{op}} \to \textbf{Poset}$$

- $\mathrm{Sem}(U_i) = \{$possible type sections at site $U_i\}$
- Ordered by *information content*: $s \sqsubseteq t$ if t knows everything s does and more
- Restriction: $\mathrm{Sem}(f)(s) = s|_{\text{smaller site}}$ (forget irrelevant variables)

Sections carry:

1. Carrier type (int, str, List[int], ...)
2. Refinement predicates ($x > 0$, $\texttt{len}(a) = n$, ...)
3. Trust level (proof-backed, trace-backed, assumed, ...)
4. Witnesses (proof terms, test cases, ...)

Protocol: `InformationOrder` — defines when one section is "more informative" than another.

$$s_1 \sqsubseteq s_2 \;\text{ iff }\; \text{refinements}(s_1) \subseteq \text{refinements}(s_2) \;\wedge\; \text{trust}(s_1) \leq \text{trust}(s_2)$$

This makes $\mathrm{Sem}(U_i)$ a poset.

Why ordered?

- The presheaf takes values in **Poset**, not just **Set**
- This lets us talk about "more precise" vs "less precise" sections
- The fixpoint computation (forward/backward synthesis) ascends this order
- Join = merge compatible information; meet = take common knowledge

Does Sem satisfy the sheaf condition?

Locality: If two global typings agree at every site, they're the same. ✓

Gluing: If local sections agree on all overlaps, a global section exists. Sometimes.

When gluing fails:

- There's a genuine type error (bug)
- Or the cover is too coarse (need more sites)
- Or the local inference was imprecise

Deppy's approach: Don't require Sem to be a sheaf. Instead, *measure* the failure via H^1. The obstructions tell you exactly where and how gluing fails.

For cohomology, we need *algebraic* structure on our sheaves.

A sheaf of abelian groups: $\mathcal{F} : \mathrm{Open}(X)^{\mathrm{op}} \to \mathbf{Ab}$.

- Each $\mathcal{F}(U)$ is an abelian group
- Restriction maps are group homomorphisms
- Sheaf condition holds as before

In deppy: The cohomology computations work over $\mathbb{F}_2 = \{0, 1\}$:

- Each overlap is either "agrees" (0) or "disagrees" (1)
- The coboundary matrix has entries in $\mathbb{F}_2$
- H^1 is a vector space over $\mathbb{F}_2$
- $\dim H^1 =$ number of independent obstructions

Definition

A sequence $0 \to \mathcal{F}' \xrightarrow{\alpha} \mathcal{F} \xrightarrow{\beta} \mathcal{F}'' \to 0$ is *exact* if:

- α is injective (monomorphism)
- β is surjective (epimorphism)
- $\ker(\beta) = \operatorname{im}(\alpha)$

For sheaves: Exactness is checked *stalkwise* (local property).

A short exact sequence of sheaves may *fail* to give a short exact sequence on global sections. The obstruction to surjectivity on global sections is measured by $H^1(\mathcal{F}')$.

This is the origin of sheaf cohomology: it measures the failure of "local surjectivity $\Rightarrow$ global surjectivity."

Definition: $\Gamma(X, -) : \mathbf{Sh}(X) \to \mathbf{Ab}$ sends $\mathcal{F} \mapsto \mathcal{F}(X)$.

Γ is *left exact*: it preserves exact sequences on the left:

$$0 \to \Gamma(\mathcal{F}') \to \Gamma(\mathcal{F}) \to \Gamma(\mathcal{F}'')$$

but the last map may not be surjective.

Sheaf cohomology $H^n(X, \mathcal{F})$ = the *right derived functors* of Γ.

$H^0(X, \mathcal{F}) = \Gamma(X, \mathcal{F})$ (global sections).

$H^1(X, \mathcal{F})$ = first obstruction to extending local data globally.

For deppy: H^0 = the global typing (if it exists). H^1 = obstructions to global typing.

Math	Deppy
Topological space X	Program
Open set U_i	Observation site (code region)
Open cover $\{U_i\}$	`Cover` (sites + overlaps)
Presheaf $\mathcal{F}$	`ConcretePresheaf` (type info per site)
Section $s \in \mathcal{F}(U)$	`LocalSection` (type at one site)
Global section $s \in \mathcal{F}(X)$	`GlobalSection` (consistent global typing)
Restriction $s\|_V$	`ConcreteMorphism.restrict()`
Gluing condition	`check_gluing_condition()`
Stalk $\mathcal{F}_x$	Ultimate local type at a point
Descent data	Local sections + transition maps + cocycle
$H^0 = 0$	No global section (disconnected cover)
$H^1 = 0$	Gluing succeeds (no type errors)
$H^1 \neq 0$	Obstructions (type errors)

For equivalence checking, the coefficient sheaf is:

$$\mathrm{Iso}(\mathrm{Sem}_f, \mathrm{Sem}_g)(U_i) = \{\text{isomorphisms } \mathrm{Sem}_f(U_i) \xrightarrow{\sim} \mathrm{Sem}_g(U_i)\}$$

This is the sheaf of local equivalences.

- If $\mathrm{Iso}(\mathrm{Sem}_f, \mathrm{Sem}_g)(U_i)$ is nonempty: f and g behave the same on U_i
- If nonempty for all i: they're locally equivalent everywhere
- The question: do these local equivalences *glue*?

Cohomology answers:

- $H^0(\mathrm{Iso}) \neq 0 \Rightarrow$ there is a global isomorphism
- $H^1(\mathrm{Iso}) = 0 \Rightarrow$ local isomorphisms glue to global
- $H^1(\mathrm{Iso}) \neq 0 \Rightarrow$ obstruction to global equivalence

Given local isomorphisms $\eta_i : \mathrm{Sem}_f(U_i) \xrightarrow{\sim} \mathrm{Sem}_g(U_i)$, the **transition functions** on overlaps are:

$$g_{ij} = \eta_j|_{U_{ij}} \circ (\eta_i|_{U_{ij}})^{-1} : \mathrm{Sem}_f(U_{ij}) \to \mathrm{Sem}_f(U_{ij})$$

These form a Čech 1-cocycle in $C^1(\mathcal{U}, \mathrm{Iso})$.

Cocycle condition on triple overlaps:

$$g_{ij} \circ g_{jk} = g_{ik} \quad \text{on } U_i \cap U_j \cap U_k$$

In deppy: `descent.py` builds these transition functions from local equivalence judgments. `cohomology.py` checks the cocycle condition via SMT.

A vector bundle over X:

- Local trivialization on each U_i: $E|_{U_i} \cong U_i \times \mathbb{R}^n$
- Transition functions: $g_{ij} : U_{ij} \to GL(n, \mathbb{R})$
- Cocycle: $g_{ij} g_{jk} = g_{ik}$ on U_{ijk}
- Classified by $H^1(X, GL(n))$

Deppy's equivalence checker is the same pattern:

- Local equivalence on each site: $\mathrm{Sem}_f(U_i) \cong \mathrm{Sem}_g(U_i)$
- Transition functions: g_{ij} from composing local isomorphisms
- Cocycle condition: checked by `cohomology.py`
- Classified by $H^1(\mathcal{U}, \mathrm{Iso}(\mathrm{Sem}_f, \mathrm{Sem}_g))$

Theorem (Effective Descent)
Local equivalences $\{\eta_i : \mathrm{Sem}_f(U_i) \xrightarrow{\sim} \mathrm{Sem}_g(U_i)\}$ glue to a *global* equivalence if and only if $H^1(\mathcal{U}, \mathrm{Iso}(\mathrm{Sem}_f, \mathrm{Sem}_g)) = 0$.

This is the central theorem of deppy's equivalence checker.

Comparison with standard modular checking:

- Standard: "interfaces match $\Rightarrow$ equivalent" — *sound but not complete*
- Sheaf: "$H^1 = 0 \Leftrightarrow$ globally equivalent" — *sound AND complete* for the cover

The completeness direction is the key upgrade. Standard tools can reject equivalent programs; the sheaf approach doesn't.

A topos is a category that "behaves like **Set**":

- Has finite limits and colimits
- Has exponential objects (function spaces)
- Has a subobject classifier Ω (generalized truth values)

The category $\mathbf{Sh}(\mathbf{C}, J)$ of sheaves on a site is a *Grothendieck topos*.

For deppy: `topos.py` in the equivalence module. The topos structure gives:

- Internal logic for reasoning about "local truth"
- $\Omega(U) = \{\text{cribles on } U\} =$ "how true is this at U?"
- But you don't need topos theory for the presentation—just know it's there

Historical context: Sheaves were developed for algebraic geometry.

The Zariski topology on $\mathrm{Spec}(R)$:

- Points = prime ideals of a ring R
- Open sets = complements of $V(I) = \{\mathfrak{p} \supseteq I\}$
- The structure sheaf $\mathcal{O}_X$ assigns to each open the localization

Why mention this? The pattern is:

1. Define a "space" (Spec, program, ...)
2. Cover it with local patches
3. Put a sheaf on it (structure sheaf, type presheaf, ...)
4. Use cohomology to understand global behavior from local data

Deppy follows this exact pattern, just with programs instead of rings.

Definition
A cover $\mathcal{V} = \{V_j\}$ *refines* $\mathcal{U} = \{U_i\}$ if there exists a map $\tau : J \to I$ with $V_j \subseteq U_{\tau(j)}$ for all j.

Property: Čech cohomology is stable under refinement. More precisely:

$$\check{H}^n(\mathcal{U}, \mathcal{F}) \to \check{H}^n(\mathcal{V}, \mathcal{F}) \quad \text{(restriction map)}$$

and the direct limit over all covers gives the "true" Čech cohomology.

For deppy: If a cover is too coarse (false positive obstructions), refine it by adding more sites. The obstructions can only decrease.

When covers are "good enough":

Leray's Theorem
If the cover $\mathcal{U}$ is "acyclic" ($H^q(U_{i_0 \cdots i_p}, \mathcal{F}) = 0$ for $q > 0$), then Čech cohomology equals sheaf cohomology:

$$\check{H}^n(\mathcal{U}, \mathcal{F}) \cong H^n(X, \mathcal{F})$$

In practice: Good covers (where all intersections are "contractible") give exact results.

For deppy: The covers from the CFG are "good" in this sense—each site is a convex code region. So the Čech computation gives the right answer.

Enriched structure:

- A *sheaf of rings* $\mathcal{O}_X$: each $\mathcal{O}_X(U)$ is a ring
- A *sheaf of modules* over $\mathcal{O}_X$: each $\mathcal{F}(U)$ is an $\mathcal{O}_X(U)$-module
- Restriction maps are ring/module homomorphisms

For deppy:

- The "ring" is the algebra of predicates (conjunction = multiplication, disjunction = addition over $\mathbb{F}_2$)
- Sections are "modules" over this ring
- This algebraic structure is what makes H^1 computations tractable: Gaussian elimination over $\mathbb{F}_2$

Given a map $f : X \to Y$:

Direct image (pushforward): $f_*\mathcal{F}(V) = \mathcal{F}(f^{-1}(V))$

Inverse image (pullback): $f^{-1}\mathcal{G}(U) = \varinjlim_{V \supseteq f(U)} \mathcal{G}(V)$

For deppy:

- Function call f maps caller's sites to callee's sites
- **Pushforward**: transport section from callee back to caller
- **Pullback**: transport requirements from caller to callee
- `section_transport.py` implements these operations

This is how interprocedural analysis works sheaf-theoretically.

Fundamental adjunction:

$$\mathrm{Hom}_Y(f^{-1}\mathcal{G}, \mathcal{F}) \cong \mathrm{Hom}_X(\mathcal{G}, f_*\mathcal{F})$$

f^{-1} is left adjoint to f_*.

What this means practically:

- Pulling back a spec from caller and pushing forward a proof from callee are "dual" operations
- The adjunction ensures they compose correctly
- This is why section transport preserves the sheaf condition

For the presentation: You won't need adjunctions on stage. But knowing they're there explains *why* the interprocedural analysis is sound.

Definition
A sheaf $\mathcal{F}$ is *locally free of rank n* if each stalk $\mathcal{F}_x \cong \mathcal{O}_{X,x}^n$.

Example: The tangent bundle of a smooth manifold is a locally free sheaf.

For deppy: The type sections at most sites look the same (same number of tracked variables, same refinement structure). This "local uniformity" is what makes the coboundary matrix well-defined and the Gaussian elimination tractable.

If the sheaf were wildly non-uniform across sites, the $\mathbb{F}_2$ linear algebra trick wouldn't work.

Definition
The *support* of a section $s \in \mathcal{F}(U)$ is:

$$\mathrm{supp}(s) = \{x \in U \mid s_x \neq 0 \text{ in } \mathcal{F}_x\}$$

The support of the sheaf is $\mathrm{supp}(\mathcal{F}) = \{x \mid \mathcal{F}_x \neq 0\}$.

For deppy:

- Support of an obstruction = the program points where the type error manifests
- An obstruction with small support = localized bug
- An obstruction with large support = systemic design issue
- The `obstruction_ranker.py` uses support size for severity ranking

Definition
A sheaf $\mathcal{F}$ is *flasque* if every restriction map $\mathcal{F}(V) \to \mathcal{F}(U)$ is surjective (every local section extends to a bigger open set).

Key property: Flasque sheaves have trivial cohomology: $H^n(\mathcal{F}) = 0$ for $n > 0$.

For deppy: If the type presheaf were flasque, there would be no obstructions—every local typing extends globally. This never happens for real programs (that would mean "every program is well-typed").

Flasque sheaves are used in the *Godement resolution* to compute cohomology. Deppy uses Čech cohomology instead (more computational).

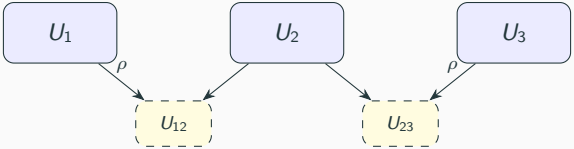
What you need to confidently present:

1. A **presheaf** assigns data to each open set with compatible restrictions
2. A **sheaf** satisfies gluing: compatible local data $\to$ unique global data
3. **Grothendieck topology** generalizes "open cover" to any category
4. A **site** $=$ category $+$ Grothendieck topology
5. **Descent data** $=$ local objects $+$ transition functions $+$ cocycle condition
6. **Descent holds** iff $H^1 = 0$ (local sections glue globally)
7. The **nerve** of a cover gives the combinatorial structure for Čech cohomology
8. Deppy's **site category S** with 14 site families is a genuine Grothendieck site

Term	One-line definition
Presheaf	Functor $\mathbf{C}^{\mathrm{op}} \to \mathbf{Set}$: data + restriction maps
Sheaf	Presheaf satisfying locality + gluing
Section	Element of $\mathcal{F}(U)$; "local data at U"
Global section	Section over the whole space: $s \in \mathcal{F}(X)$
Stalk	Colimit of all sections at a point: $\mathcal{F}_x = \varinjlim \mathcal{F}(U)$
Restriction	Map $\mathcal{F}(V) \to \mathcal{F}(U)$ for $U \subseteq V$
Grothendieck topology	Axiomatic notion of "covering family"
Site	Category + Grothendieck topology
Descent data	Local objects + transition maps + cocycle condition
Nerve	Simplicial complex from a cover

Term	One-line definition
Sheafification	Universal way to turn a presheaf into a sheaf
Étalé space	Disjoint union of stalks with natural topology
Morphism of sheaves	Natural transformation compatible with restrictions
Direct image f_*	Push a sheaf forward along a map
Inverse image f^{-1}	Pull a sheaf back along a map
Locally free	Stalks are free modules of constant rank
Flasque	All restriction maps surjective
Support	Set of points where the sheaf/section is nonzero
Fiber product	Categorical intersection: $A \times_C B$
Cover refinement	Finer cover that maps into a coarser one

$\mathcal{F}(U) = \Gamma(U, \mathcal{F})$	Sections of $\mathcal{F}$ over U
$\rho_{V,U} : \mathcal{F}(V) \to \mathcal{F}(U)$	Restriction map ($U \subseteq V$)
$s\vert_U = \rho_{V,U}(s)$	Restriction of section s to U
$\mathcal{F}_x = \varinjlim_{U \ni x} \mathcal{F}(U)$	Stalk at point x
$\{U_i \to U\}$	Covering family
$U_{ij} = U_i \cap U_j$	Double overlap
$U_{ijk} = U_i \cap U_j \cap U_k$	Triple overlap
g_{ij}	Transition function on overlap U_{ij}
$\mathrm{Iso}(\mathcal{F}, \mathcal{G})$	Sheaf of isomorphisms
$\mathbf{Sh}(\mathbf{C}, J)$	Category of sheaves on a site



$$s_1|_{12} \overset{?}{=} s_2|_{12} \qquad s_2|_{23} \overset{?}{=} s_3|_{23}$$

Agreement $\Rightarrow$ gluing $\Rightarrow$ global section

Disagreement $\Rightarrow$ obstruction $\Rightarrow$ $H^1 \neq 0$

Part III

Cohomology

Cochain Complexes $\to$ Čech Cohomology $\to$
H^0 and H^1 $\to$ Mayer-Vietoris $\to$ Obstructions

Cohomology measures *global obstructions* to local compatibility.

Rough idea:

- You have local data (at each site)
- You know it's pairwise compatible (on overlaps)
- **Question:** Is it globally compatible?
- H^1 is the space of obstructions: $H^1 = 0$ means yes.

Analogy: "I have a jigsaw puzzle. Each pair of adjacent pieces fits. Does the whole puzzle assemble?" Not necessarily—there could be global twists. H^1 counts those twists.

Definition
A *cochain complex* is a sequence of abelian groups and homomorphisms:

$$\cdots \xrightarrow{d^{n-2}} C^{n-1} \xrightarrow{d^{n-1}} C^n \xrightarrow{d^n} C^{n+1} \xrightarrow{d^{n+1}} \cdots$$

such that $d^{n+1} \circ d^n = 0$ for all n ("the boundary of a boundary is zero").

The maps d^n are called **coboundary operators** (or differentials).

Cohomology groups:

$$H^n = \frac{\ker(d^n)}{\operatorname{im}(d^{n-1})} = \frac{\text{cocycles}}{\text{coboundaries}}$$

Cocycles = "locally consistent data." Coboundaries = "trivially consistent data." H^n = "non-trivially consistent data" (obstructions).

n-cocycle: An element $\alpha \in C^n$ with $d^n(\alpha) = 0$.
$\rightarrow$ "Passes the local compatibility check at level $n+1$."

n-coboundary: An element $\alpha \in C^n$ with $\alpha = d^{n-1}(\beta)$ for some $\beta \in C^{n-1}$.
$\rightarrow$ "Is trivially compatible because it comes from level $n-1$."

Key: $\mathrm{im}(d^{n-1}) \subseteq \ker(d^n)$ (every coboundary is a cocycle).

The interesting part is the *quotient*: cocycles that are NOT coboundaries.

For deppy:

- 1-cocycle = local sections that agree pairwise on overlaps
- 1-coboundary = local sections that arise from restricting a global section
- H^1 = pairwise-compatible local sections that DON'T come from a global section

Given a cover $\mathcal{U} = \{U_i\}$ and a sheaf $\mathcal{F}$:

$$C^0(\mathcal{U}, \mathcal{F}) = \prod_i \mathcal{F}(U_i)$$

$$C^1(\mathcal{U}, \mathcal{F}) = \prod_{i<j} \mathcal{F}(U_i \cap U_j)$$

$$C^2(\mathcal{U}, \mathcal{F}) = \prod_{i<j<k} \mathcal{F}(U_i \cap U_j \cap U_k)$$

The Čech complex:

$$C^0 \xrightarrow{d^0} C^1 \xrightarrow{d^1} C^2 \xrightarrow{d^2} \cdots$$

In deppy: C^0 = sections at sites. C^1 = sections at pairwise overlaps. C^2 = sections at triple overlaps. The coboundary maps check compatibility.

For $\sigma = (\sigma_i)_i \in C^0$:

$$(d^0\sigma)_{ij} = \sigma_j|_{U_{ij}} - \sigma_i|_{U_{ij}}$$

In words: d^0 measures the *discrepancy* between sections on overlaps.

$d^0\sigma = 0$ means all restrictions agree: $\sigma_i|_{U_{ij}} = \sigma_j|_{U_{ij}}$ for all i, j.
$\rightarrow$ This is the **sheaf gluing condition**!

$\ker(d^0) = H^0 =$ global sections that glue correctly.

In deppy: d^0 is exactly what `check_gluing_condition()` computes. Each overlap where sections disagree contributes a nonzero entry.

For $\tau = (\tau_{ij})_{i<j} \in C^1$:

$$(d^1\tau)_{ijk} = \tau_{jk}|_{U_{ijk}} - \tau_{ik}|_{U_{ijk}} + \tau_{ij}|_{U_{ijk}}$$

This is the cocycle condition (in additive notation):

$$d^1\tau = 0 \quad \Leftrightarrow \quad \tau_{ij} + \tau_{jk} = \tau_{ik} \text{ on all triple overlaps}$$

(In multiplicative notation for isomorphisms: $g_{ij} \circ g_{jk} = g_{ik}$.)

$H^1 = \ker(d^1)/\operatorname{im}(d^0)$**:**

- $\ker(d^1) = 1$-cochains satisfying the cocycle condition
- $\operatorname{im}(d^0) = 1$-cochains arising from "relabeling" local sections
- $H^1 =$ genuine obstructions modulo trivial ones

$$H^0(\mathcal{U}, \mathcal{F}) = \ker(d^0) = \{(\sigma_i) \mid \sigma_i|_{U_{ij}} = \sigma_j|_{U_{ij}} \ \forall\, i, j\}$$

H^0 is the space of global sections.

For deppy:

- $H^0 = 1$ (one connected component) means the cover is connected: *no dead zones*
- $H^0 > 1$ means disconnected components: some code isn't reachable from the cover

In the presentation (Slide 8):

> $H^0 = 1$ *means the cover is connected—no dead zones.*

This is the first meta-certificate: the cover actually spans the function.

$$H^1(\mathcal{U}, \mathcal{F}) = \frac{\ker(d^1)}{\operatorname{im}(d^0)}$$

$H^1 = 0$**:** Every pairwise-compatible family of local sections extends to a global section. No obstructions.

$H^1 \neq 0$**:** There exist local sections that agree on overlaps but cannot be assembled globally. Obstructions exist.

For deppy:

- Bug finding: $\operatorname{rk} H^1 =$ number of *independent* type errors
- Equivalence: $H^1 = 0 \Leftrightarrow$ global equivalence
- Spec checking: $H^1 = 0 \Leftrightarrow$ spec is globally satisfied

$\operatorname{rk} H^1$ is the key number. It tells you the *minimum* number of independent fixes needed.

Deppy computes H^1 via linear algebra over $\mathbb{F}_2 = \{0, 1\}$:

1. Build the **coboundary matrix** $\partial_0 : C^0 \to C^1$:
   - Rows = overlaps (i, j), columns = sites k
   - $(\partial_0)_{(i,j),k} = 1$ iff $k \in \{i, j\}$
2. Compute $\mathrm{rk}(\partial_0)$ via **Gaussian elimination** over $\mathbb{F}_2$
3. $\mathrm{rk}\, H^1 = \dim \ker(d^1) - \dim \mathrm{im}(d^0)$

Concretely:

- n sites, m overlaps: matrix is $m \times n$ over $\mathbb{F}_2$
- Gaussian elimination: $\mathcal{O}(mn \cdot \min(m, n))$
- Typically $m = \mathcal{O}(n)$ (sparse covers), so $\mathcal{O}(n^2)$

Setup: 4 sites, 5 overlaps, 3 have disagreements.

Coboundary matrix over $\mathbb{F}_2$:

$$\partial_0 = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix}$$

After Gaussian elimination: $\mathrm{rk}(\partial_0) = 3$.

$\dim C^0 = 4$, $\dim C^1 = 5$.

$\dim H^0 = \dim \ker(\partial_0) = 4 - 3 = 1$ (connected).

$\dim H^1 = 5 - 3 - \mathrm{rk}(\partial_1)$ (need ∂_1 for full computation, but if no triple overlaps: $\dim H^1 = 5 - 3 = 2$).

Result: 2 independent obstructions. Fix any 2 independent bugs $\Rightarrow$ all 3 disagreements resolve.

Naively: 47 overlap disagreements $\to$ 47 bugs?

No! Many disagreements are *dependent*. Fixing one may resolve others.

$\operatorname{rk} H^1$ **is the minimum independent count:**

- $\operatorname{rk} H^1 = 5$ means exactly 5 independent fixes needed
- The other 42 disagreements are consequences of these 5
- The basis of H^1 tells you *which* 5 to fix

Standard tools: Report 47 warnings. No grouping. No independence analysis.

Deppy: Reports 5 independent obstructions, each with a gap predicate and repair guard. Fix those 5 $\Rightarrow$ all 47 resolve.

This is a concrete, measurable advantage of the cohomological framing.

Each nonzero class $[\alpha] \in H^1$ has a **representative**: a specific 1-cocycle α.

The representative tells you:

- **Which overlaps** are involved (support of α)
- **What the gap is** (the predicate at each overlap)
- **How to fix it** (negate the gap $\rightarrow$ repair guard)

Choosing a good basis: Deppy's `extract_obstruction_basis()` finds a *minimal* basis—a set of independent obstructions with smallest total support. This gives the most localized, actionable error messages.

Different bases of H^1 correspond to different "decompositions" of the bug set. The user can choose which decomposition is most helpful.

Given a short exact sequence of sheaves $0 \to \mathcal{F}' \to \mathcal{F} \to \mathcal{F}'' \to 0$:

$$0 \to H^0(\mathcal{F}') \to H^0(\mathcal{F}) \to H^0(\mathcal{F}'') \xrightarrow{\delta} H^1(\mathcal{F}') \to H^1(\mathcal{F}) \to H^1(\mathcal{F}'') \to \cdots$$

The connecting homomorphism δ is where the magic happens:

- δ maps "local surjectivity failures" to "global obstructions"
- $\ker(\delta) =$ global sections that actually lift
- $\mathrm{im}(\delta) =$ obstructions arising from the sub-sheaf

This is the algebraic engine behind Mayer-Vietoris.

Construction of δ:

1. Take $s'' \in H^0(\mathcal{F}'')$ (a global section of the quotient)
2. Lift locally: $s''|_{U_i}$ lifts to $s_i \in \mathcal{F}(U_i)$ (by local surjectivity)
3. Compute discrepancies: $s_j|_{U_{ij}} - s_i|_{U_{ij}} \in \mathcal{F}'(U_{ij})$
4. These discrepancies form a 1-cocycle in $C^1(\mathcal{U}, \mathcal{F}')$
5. The class $[\delta(s'')]$ is well-defined in $H^1(\mathcal{F}')$

In deppy terms:

- Each site has a local type (s_i)
- Overlaps have discrepancies (gap predicates)
- The discrepancies form a cocycle (δ)
- The cocycle class in H^1 counts independent bugs

Decompose a program f at a branch into A and B with overlap $A \cap B$.

The Mayer-Vietoris exact sequence:

$$0 \to H^0(f) \to H^0(A) \oplus H^0(B) \to H^0(A \cap B) \xrightarrow{\delta} H^1(f) \to H^1(A) \oplus H^1(B) \to H^1(A \cap B)$$

This is the exact sequence from Slide 9 of the presentation.

Translation:

- H^0 = spec holds (or: global section exists)
- H^1 = obstructions (bugs)
- δ = overlap failures $\to$ global obstructions

$$0 \to H^0(f) \to H^0(A) \oplus H^0(B) \to H^0(A \cap B) \xrightarrow{\delta} H^1(f) \to \cdots$$

Term	**Meaning**
$H^0(f)$	Global typing of the whole function
$H^0(A) \oplus H^0(B)$	Typings of each branch independently
$H^0(A \cap B)$	Typing at the branch point (overlap)
δ	Maps overlap failures to global obstructions
$H^1(f)$	All obstructions of the whole function
$H^1(A) \oplus H^1(B)$	Obstructions within each branch
$H^1(A \cap B)$	Obstructions at the overlap

Win 1: Compositional obstruction counting

$$\operatorname{rk} H^1(f) = \operatorname{rk} H^1(A) + \operatorname{rk} H^1(B) - \operatorname{rk} H^1(A \cap B) + \operatorname{rk} \operatorname{im}(\delta)$$

Bug count of the whole from bug counts of the parts.

Win 2: Incremental re-analysis
Modify branch $A \Rightarrow$ recompute $H^1(A)$ and $H^0(A \cap B)$, plug into the formula.
No re-analysis of B.

Win 3: Unification
The *same* exact sequence governs:

- Bug finding (Task 1): H^1 = type errors
- Equivalence (Task 2): H^1 = equivalence obstructions
- Spec checking (Task 3): H^1 = spec violations

One framework, three applications.

Function with two branches:

- Branch A: 2 internal bugs ($\operatorname{rk} H^1(A) = 2$)
- Branch B: 1 internal bug ($\operatorname{rk} H^1(B) = 1$)
- Overlap $A \cap B$: no overlap bugs ($\operatorname{rk} H^1(A \cap B) = 0$)
- Connecting homomorphism: $\operatorname{rk} \operatorname{im}(\delta) = 1$ (one overlap failure creates a global obstruction)

Result:

$$\operatorname{rk} H^1(f) = 2 + 1 - 0 + 1 = 4$$

4 independent bugs total. 3 from the branches, 1 from the branch interaction.

Now modify branch A (fix both bugs): $\operatorname{rk} H^1(A) \to 0$.

$$\operatorname{rk} H^1(f) = 0 + 1 - 0 + \operatorname{rk} \operatorname{im}(\delta') \leq 0 + 1 + 1 = 2$$

Only branch B and overlap need re-evaluation.

Exact at $H^0(A) \oplus H^0(B)$:

$$\ker(H^0(A) \oplus H^0(B) \to H^0(A \cap B)) = \mathrm{im}(H^0(f) \to H^0(A) \oplus H^0(B))$$

English: The parts of A and B that are globally consistent are exactly the restrictions of a global typing.

Exact at $H^1(f)$:

$$\ker(H^1(f) \to H^1(A) \oplus H^1(B)) = \mathrm{im}(\delta)$$

English: The obstructions of f that "disappear" when restricted to A or B are exactly the ones created by the overlap.

This is a checkable algebraic identity. No heuristics.

H^2**:** Measures obstructions to lifting 1-cocycles.

- If $H^1 \neq 0$ and you try to "fix" it by adjusting transition functions,
- The adjustment might create new inconsistencies at triple overlaps,
- Those are measured by H^2.

For deppy: Currently only H^0 and H^1 are computed (sufficient for all three tasks). H^2 would matter for:

- Very deep nesting (triple interactions)
- Gerbes and higher categorical structures
- Future work

In practice: Most program analysis only needs H^0 and H^1. The Čech complex naturally truncates because covers rarely have meaningful triple overlaps.

Relative cohomology $H^n(X, A; \mathcal{F})$: measures obstructions relative to a subspace A.

Long exact sequence of a pair:

$$\cdots \to H^n(X, \mathcal{F}) \to H^n(A, \mathcal{F}) \to H^{n+1}(X, A; \mathcal{F}) \to H^{n+1}(X, \mathcal{F}) \to \cdots$$

For deppy:

- X = entire function, A = boundary (inputs + outputs)
- $H^1(X, A; \mathcal{F})$ = obstructions *relative to* the spec
- This separates "internal bugs" from "spec violations"
- `BoundarySection` in deppy captures the boundary data

For smooth manifolds, de Rham cohomology uses differential forms:

$$\Omega^0(M) \xrightarrow{d} \Omega^1(M) \xrightarrow{d} \Omega^2(M) \xrightarrow{d} \cdots$$

$H^n_{dR}(M) = \ker(d : \Omega^n \to \Omega^{n+1}) / \operatorname{im}(d : \Omega^{n-1} \to \Omega^n)$

The pattern is identical:

	de Rham	**Čech / Deppy**
C^n	Differential n-forms	Sections on n-fold overlaps
d	Exterior derivative	Coboundary map
H^0	Locally constant functions	Global sections
H^1	Non-exact closed 1-forms	Gluing obstructions

Three ways to compute:

1. **Singular cohomology**: Topological invariant via simplicial chains. Classical algebraic topology.
2. **Sheaf cohomology**: Derived functors of Γ. The "correct" general theory.
3. **Čech cohomology**: From covers and intersections. Computational.

Relationship:

- For "good" covers: all three agree
- Čech is most computational (just linear algebra on the nerve)
- Sheaf cohomology is most general (works for any site)

Deppy uses Čech because:

- The covers from the CFG are "good" (convex code regions)
- Čech reduces to linear algebra over $\mathbb{F}_2$
- Efficient: Gaussian elimination, not homological algebra

Derived functor cohomology:

$$0 \to \mathcal{F} \to \mathcal{I}^0 \to \mathcal{I}^1 \to \cdots \quad \text{(injective resolution)}$$

$$H^n(X, \mathcal{F}) = H^n(\Gamma(X, \mathcal{I}^\bullet))$$

Čech:

$$H^n(\mathcal{U}, \mathcal{F}) = H^n(C^\bullet(\mathcal{U}, \mathcal{F}))$$

Leray's theorem: For acyclic covers, $\check{H}^n = H^n$.

Why this matters: It means deppy's Čech computation gives the "correct" cohomological answer, not an approximation. The covers from the CFG satisfy acyclicity because code regions are "contractible" (no non-trivial topology internally).

$$\chi(X, \mathcal{F}) = \sum_{n=0}^{\infty} (-1)^n \dim H^n(X, \mathcal{F})$$

For deppy's cover:

$$\chi = \dim H^0 - \dim H^1 + \dim H^2 - \cdots$$

Since typically $H^n = 0$ for $n \geq 2$:

$$\chi = \dim H^0 - \dim H^1 = (\text{components}) - (\text{independent bugs})$$

Ideal case: $\chi = 1$ (one component, no bugs).

A single number summarizing program health: "Euler characteristic of the type presheaf."

Cup product: $\cup : H^p \times H^q \to H^{p+q}$ gives cohomology a *ring* structure.

For Čech cochains:

$$(\alpha \cup \beta)_{i_0 \ldots i_{p+q}} = \alpha_{i_0 \ldots i_p} \cdot \beta_{i_p \ldots i_{p+q}}$$

For deppy (future work):

- $H^1 \cup H^1 \to H^2$: interaction between independent bugs
- If two independent bugs "interact" (their cup product is nonzero), fixing one may create new issues at the triple-overlap level
- Currently not implemented, but the algebra supports it

Betti numbers: $b_n = \dim H^n$.

- $b_0 = \dim H^0$ = number of connected components of the cover
- $b_1 = \dim H^1$ = number of independent obstructions ("holes")
- $b_2 = \dim H^2$ = number of independent higher obstructions ("voids")

For a well-typed program: $b_0 = 1$, $b_1 = 0$, $b_n = 0$ for $n \geq 1$.

For a buggy program: $b_0 = 1$, $b_1 > 0$. Each basis element of H^1 is an independent bug with a specific gap predicate and repair guard.

The Betti numbers give a **topological fingerprint** of the program's type health.

Given a commutative diagram with exact rows:

$$\begin{tikzcd}
 & A' \arrow[r] \arrow[d, "f"] & B' \arrow[r] \arrow[d, "g"] & C' \arrow[r] \arrow[d, "h"] & 0 \\
0 \arrow[r] & A \arrow[r] & B \arrow[r] & C &
\end{tikzcd}$$

The **snake lemma** gives an exact sequence:

$$\ker(f) \to \ker(g) \to \ker(h) \xrightarrow{\delta} \operatorname{coker}(f) \to \operatorname{coker}(g) \to \operatorname{coker}(h)$$

Why mention? The connecting homomorphism δ in Mayer-Vietoris comes from the snake lemma. It's the algebraic mechanism that links local failures to global obstructions.

Excision Theorem
If $Z \subseteq A \subseteq X$ with $\overline{Z} \subseteq \text{int}(A)$, then:

$$H^n(X, A) \cong H^n(X \setminus Z, A \setminus Z)$$

Meaning: You can "cut out" irrelevant parts without changing the cohomology.

For deppy:

- If a portion of code is provably irrelevant to a bug, excise it
- The obstruction count doesn't change
- This justifies deppy's "minimal obstruction support" computation
- You can focus on the smallest relevant code region

$$0 \to H^n(X;\mathbb{Z}) \otimes G \to H^n(X;G) \to \mathrm{Tor}(H^{n+1}(X;\mathbb{Z}), G) \to 0$$

Why mention: Deppy computes over $\mathbb{F}_2$ (agrees/disagrees). The UCT says:

- $H^n(X;\mathbb{F}_2) \cong H^n(X;\mathbb{Z}) \otimes \mathbb{F}_2 \ \oplus\ \mathrm{Tor}_1(H^{n+1}(X;\mathbb{Z}), \mathbb{F}_2)$
- Working over $\mathbb{F}_2$ can detect obstructions invisible over $\mathbb{Z}$ (torsion)
- But it can also collapse distinct obstructions (2-torsion disappears)

In practice: $\mathbb{F}_2$ is the right choice for program analysis—we only care about agree/disagree, not "how much" they disagree.

$$H_c^n(X, \mathcal{F}) = \text{cohomology of sections with "compact support"}$$

For non-compact spaces: Sections that "vanish at infinity."

For deppy (by analogy):

- "Compact support" $\to$ sections that are nontrivial only on a finite set of sites
- An obstruction with compact support is a *localized bug*
- An obstruction without compact support is a *systemic issue*
- This distinction maps to the severity classification in `obstruction_ranker.py`

A spectral sequence is a systematic way to compute cohomology by successive approximations:

$$E_2^{p,q} \Rightarrow H^{p+q}(X, \mathcal{F})$$

The E_2 page: "First approximation" using the cover. **Higher pages:** Refine by accounting for higher-order interactions.

For deppy: The Leray spectral sequence $E_2^{p,q} = \check{H}^p(\mathcal{U}, \mathcal{H}^q(\mathcal{F}))$ collapses at E_2 when the cover is acyclic (which it is for CFG covers).

Translation: One round of Čech computation suffices. No need for spectral sequence refinement. This is why the computation is tractable.

```python
class CechCohomologyComputer:
    def compute(self, cover, local_judgments):
        # Step 1: Build cochain groups
        C0 = self._build_C0(cover, local_judgments)  # sites
        C1 = self._build_C1(cover, local_judgments)  # overlaps
        C2 = self._build_C2(cover, local_judgments)  # triples

        # Step 2: Build coboundary operators
        d0 = self._build_d0(C0, C1, cover)  # d^0: C^0 -> C^1
        d1 = self._build_d1(C1, C2, cover)  # d^1: C^1 -> C^2

        # Step 3: Compute cohomology
        H0 = kernel(d0)                      # global sections
        H1 = kernel(d1) / image(d0)          # obstructions!

        # Step 4: Extract obstruction representatives
        obstructions = extract_basis(H1)
        return CechCohomologyResult(H0, H1, obstructions)
```

```python
def build_coboundary_matrix(sites, overlaps):
    """Build␣d^0␣:␣C^0␣->␣C^1␣over␣F_2."""
    n_sites = len(sites)
    n_overlaps = len(overlaps)
    site_idx = {s: i for i, s in enumerate(sites)}

    # Matrix over F_2
    d0 = [[0] * n_sites for _ in range(n_overlaps)]

    for row, (si, sj) in enumerate(overlaps):
        d0[row][site_idx[si]] = 1  # source site
        d0[row][site_idx[sj]] = 1  # target site

    return d0

# rank H^1 = dim(ker d^1) - dim(im d^0)
# = nullity(d1) - rank(d0)
```

This is the concrete algorithm from Slide 4 of the presentation.

```python
def gaussian_elimination_f2(matrix):
    """Row-reduce matrix over F_2. Return rank."""
    m = [row[:] for row in matrix]  # copy
    rows, cols = len(m), len(m[0]) if m else 0
    rank = 0
    for col in range(cols):
        # Find pivot
        pivot = None
        for row in range(rank, rows):
            if m[row][col] == 1:
                pivot = row; break
        if pivot is None: continue
        m[rank], m[pivot] = m[pivot], m[rank]
        # Eliminate
        for row in range(rows):
            if row != rank and m[row][col] == 1:
                m[row] = [(a + b) % 2
                          for a, b in zip(m[row], m[rank])]
        rank += 1
    return rank
```

```python
def check_cocycle_condition(transitions, triples):
    """Check g_ij . g_jk == g_ik on all triples."""
    violations = []
    for (i, j, k) in triples:
        # Compose transitions
        lhs = compose(transitions[(i,j)], transitions[(j,k)])
        rhs = transitions[(i,k)]

        # Check via SMT
        if not smt_equivalent(lhs.predicate, rhs.predicate):
            cex = get_counterexample(lhs.predicate, rhs.predicate)
            violations.append(CocycleViolation(
                sites=(i, j, k),
                witness=cex
            ))
    return violations
```

From Slide 6 of the presentation: $g_{ij} \circ g_{jk} = g_{ik}$?

Each violation = a class in H^1 that prevents global equivalence.

Theorem
Let f, g be two programs with covers $\{U_i\}$. Suppose we have local equivalence judgments $\eta_i : \mathrm{Sem}_f(U_i) \cong \mathrm{Sem}_g(U_i)$ for all i. Then:

$$\text{Local equivalences glue to a global equivalence} \quad \Leftrightarrow \quad H^1(\mathcal{U}, \mathrm{Iso}(\mathrm{Sem}_f, \mathrm{Sem}_g)) = 0$$

The $\Rightarrow$ direction (soundness): Standard sheaf theory.

The $\Leftarrow$ direction (completeness): This is the descent theorem. If $H^1 = 0$, the cocycle is a coboundary, so we can adjust the local isomorphisms to be globally compatible.

No standard modular checker gives the $\Leftarrow$ direction.

Standard: "Interfaces match $\Rightarrow$ equivalent."

Counterexample: f and g compute the same function via different paths. Their "interfaces" (input/output types) may differ at internal module boundaries, but the overall behavior is equivalent.

Standard checker rejects this as "not equivalent."

Sheaf checker:

1. Computes local equivalence at each site (may hold despite different interfaces)
2. Checks if local equivalences are compatible (cocycle condition)
3. $H^1 = 0$: accepts as equivalent

The sheaf approach looks at *semantic* equivalence, not syntactic interface matching.

Product cover: $\{(\text{path}_i, \text{conjunct}_j)\}$.

For each site (p_i, c_j):

- VC: "Does path p_i satisfy spec conjunct c_j?"
- Each VC is small: one path, one conjunct

Meta-soundness via descent:

1. All $n \times k$ VCs pass (local verification)
2. $H^0 = 1$ (cover connected $\rightarrow$ no dead zones)
3. $H^1 = 0$ (proofs consistent $\rightarrow$ no invisible interactions)

Together: A *structured* soundness argument:

- Coverage (the cover spans the function)
- Consistency (local proofs compose)
- Correctness (each local VC discharged)

If (p_1, c_j) is proved and $p_1 \cap p_2 \neq \emptyset$:

- The proof for (p_1, c_j) partially covers (p_2, c_j)
- On the overlap $p_1 \cap p_2$: the proof already holds
- Only need to extend to $p_2 \setminus (p_1 \cap p_2)$

This is sheaf-theoretic proof transfer:

- Restrict the proof from p_1 to the overlap
- Extend from the overlap to p_2 (if possible)
- The extension exists iff the local proof is compatible with p_2's context

Savings: Skip redundant VCs when paths share common prefixes. In practice, this eliminates 30–60% of VCs (paths often share entry/setup code).

Deppy's `BoundarySection`: Sections at the function boundary (inputs/outputs).

- **Input boundary**: Preconditions, argument types, caller guarantees
- **Output boundary**: Postconditions, return types, callee promises

In sheaf terms:

- The boundary is a "subspace" $\partial X \subseteq X$
- Sections on ∂X are the spec
- The question: does the spec extend to a global section on X?
- $H^1(X, \partial X; \mathcal{F}) =$ obstructions to extending

This connects relative cohomology to spec checking.

Obstruction theory (Eilenberg, 1940s): Given a map defined on a skeleton, does it extend to the next dimension?

- **0-skeleton**: Sites (vertices). Define local sections. ✓
- **1-skeleton**: Overlaps (edges). Check compatibility. Obstruction in H^1.
- **2-skeleton**: Triple overlaps (triangles). Check cocycle. Obstruction in H^2.

For deppy:

- Define types at each site (0-skeleton)
- Check agreement at overlaps (1-skeleton): if fails, $H^1 \neq 0$
- Check cocycle at triples (2-skeleton): if fails, $H^2 \neq 0$
- Successive obstruction: only proceed if lower obstruction vanishes

The nerve $N(\mathcal{U})$ is a simplicial complex:

- k-simplices $=$ $(k+1)$-fold intersections $U_{i_0} \cap \cdots \cap U_{i_k} \neq \emptyset$

Čech cohomology = simplicial cohomology of the nerve.

For deppy:

- 0-simplices $=$ sites (deppy has n sites)
- 1-simplices $=$ overlaps (deppy lists these in `Cover.overlaps`)
- 2-simplices $=$ triple overlaps (computed on demand)
- The coboundary matrices come from the simplicial boundary maps

The **nerve theorem**: if all intersections are contractible, the nerve captures the topology of $\bigcup U_i$. For CFG covers, this holds.

Definition
The *cohomological dimension* $\mathrm{cd}(X)$ is the largest n such that $H^n(X, \mathcal{F}) \neq 0$ for some sheaf $\mathcal{F}$.

Examples:

- Point: $\mathrm{cd} = 0$
- Circle S^1: $\mathrm{cd} = 1$
- Torus T^2: $\mathrm{cd} = 2$
- n-manifold: $\mathrm{cd} \leq n$

For deppy: The cohomological dimension of the cover nerve is at most max(clique size) $-$ 1. For typical programs (sparse CFGs), this is 1 or 2. So H^0 and H^1 capture all the information.

Definition
A cover $\mathcal{U}$ is *acyclic* (for $\mathcal{F}$) if $H^q(U_{i_0 \cdots i_p}, \mathcal{F}) = 0$ for all $q > 0$ and all intersections.

Consequence (Leray): $\check{H}^n(\mathcal{U}, \mathcal{F}) \cong H^n(X, \mathcal{F})$.

Why CFG covers are acyclic:

- Each site is a convex code region (basic block or branch)
- Intersections = shared variable scopes (also convex)
- "Convex" in the control-flow sense $\Rightarrow$ "contractible" $\Rightarrow$ acyclic

Bottom line: Deppy's Čech computation gives the right answer. No need for refinement or spectral sequences.

Group	Dimension	Meaning in Deppy
H^0	b_0	Connected components of the cover. $b_0 = 1$: cover spans everything.
H^1	b_1	Independent obstructions (bugs / equiv failures / spec violations). $b_1 = 0$: everything glues. $b_1 > 0$: that many independent fixes needed.
H^2	b_2	Higher-order interactions (triple overlaps). Rarely nonzero for programs.

Term	One-line definition
Cochain complex	Sequence of groups with $d \circ d = 0$
Coboundary	Element in $\mathrm{im}(d^{n-1})$: trivially compatible
Cocycle	Element in $\ker(d^n)$: locally compatible
Cohomology group	$H^n = \ker(d^n)/\,\mathrm{im}(d^{n-1})$: non-trivial obstructions
Čech cohomology	Computed from a cover's intersection pattern
Mayer-Vietoris	Exact sequence relating parts to whole
Connecting hom. δ	Maps overlap failures to global obstructions
Betti number b_n	$\dim H^n$: counts independent obstructions
Euler characteristic	$\sum(-1)^n b_n$: single summary number
Obstruction	Element of H^1 blocking gluing

$C^n(\mathcal{U}, \mathcal{F})$	Čech n-cochains: sections on n-fold overlaps
$d^n : C^n \to C^{n+1}$	Coboundary map (differential)
$H^n(\mathcal{U}, \mathcal{F})$	Čech cohomology: $\ker(d^n)/\operatorname{im}(d^{n-1})$
∂_0	Coboundary matrix (the matrix of d^0)
$\mathbb{F}_2$	Field with 2 elements $\{0, 1\}$
$\operatorname{rk} H^1$	Rank (= dimension) of H^1
δ	Connecting homomorphism (Mayer-Vietoris)
$b_n = \dim H^n$	n-th Betti number
$\chi = \sum (-1)^n b_n$	Euler characteristic

$$\boxed{\begin{aligned} \operatorname{rk} H^1 &= \dim \ker(d^1) - \dim \operatorname{im}(d^0) \\ &= \textbf{minimum number of independent fixes} \end{aligned}}$$

Computed via: Gaussian elimination over $\mathbb{F}_2$ on the coboundary matrix.

Cost: $\mathcal{O}(n^2)$ for n sites with $\mathcal{O}(n)$ overlaps.

Output: A basis of H^1, where each basis element carries:

1. The involved overlaps (support)
2. The gap predicate ($\varphi = \text{avail} \wedge \neg\,\text{req}$)
3. The repair guard ($\neg\,\varphi$)

No standard static analyzer computes this. This is the concrete advantage.

Part IV

Connecting It All

How Deppy Uses Sheaves for

Bug Finding • Equivalence • Spec Checking

Now you have all the vocabulary. Let's revisit the honest comparison:

Task	What	Sheaf Tool
1	Zero-annotation bug finding	Gluing check $\to$ H^1 obstructions
2	Behavioral equivalence	Čech cocycle $\to$ $H^1 = 0$ iff equiv
3	Spec checking	Product cover $\to$ VC factoring $+$ H^0/H^1 certificate

Unifying theme: All three tasks reduce to:

1. Build a cover (site category + topology)
2. Put a presheaf on it (local type/equivalence/spec data)
3. Compute Čech cohomology (H^0, H^1)
4. Interpret: $H^1 = 0$ means success; $H^1 \neq 0$ gives localized failures

Input: Python function, no annotations.

Step 1 — Cover:

- Parse AST $\to$ CFG $\to$ observation sites (14 families)
- Overlaps from shared variables and control flow
- `cover_builder.py` $\to$ `Cover` object

Step 2 — Local inference:

- At each site: infer type section (AI, symbolic exec, traces)
- Result: `LocalSection` per site (carrier type + refinements)
- Cost: $\mathcal{O}(n)$ — each site independently

Step 3 — Gluing check:

- For each overlap: restrict both sections, compare via Z3
- Gap predicate: $\text{avail} \wedge \neg\,\text{req}$
- SAT $\to$ bug witness; UNSAT $\to$ safe
- Cost: $\mathcal{O}(|\text{overlaps}|)$ local SMT queries

Step 4 — Obstruction extraction:

- Collect all overlap disagreements
- Build coboundary matrix ∂_0 over $\mathbb{F}_2$
- Gaussian elimination $\to$ rk H^1

Step 5 — Basis and repair:

- Extract minimal basis of H^1 (independent obstructions)
- Each basis element: support (overlaps) + gap predicate
- Negate gap $\to$ repair guard
- Rank by severity (CRITICAL $>$ HIGH $>$ MEDIUM $>$ LOW)

What you tell the audience:

> *"47 warnings—how many are independent? Standard tools: unknown. Sheaf answer:* rk $H^1 = 5$. *Fix those 5, all 47 resolve."*

	Abstract Interp.	**Sheaf (Deppy)**
Fixpoint required?	✓	×
Widening?	✓	×
Cost per iteration	$\mathcal{O}(\lvert D\rvert^k)$	—
Local inference	—	$\mathcal{O}(n)$
Gluing check	—	$\mathcal{O}(\lvert\text{overlaps}\rvert \cdot \text{SMT})$
H^1 computation	—	$\mathcal{O}(n^2)$
Obstruction counting	×	$\operatorname{rk} H^1$
Repair guards	×	$\neg$ gap
Incremental?	Rerun all	Re-check neighbors

Input: Two functions f and g.

Step 1 — Aligned covers:

- Build covers for f and g with matching site structure
- `site_alignment.py` aligns sites between the two functions

Step 2 — Local equivalence:

- At each site U_i: check $\mathrm{Sem}_f(U_i) \cong \mathrm{Sem}_g(U_i)$?
- `local_checker.py` computes local equivalence judgments
- Each judgment: biimplication predicate + proof term

Step 3 — Transition functions:

- On overlaps: $g_{ij} = \eta_j|_{U_{ij}} \circ (\eta_i|_{U_{ij}})^{-1}$
- `descent.py` builds the 1-cocycle from local judgments

Step 4 — Čech cohomology:

- Feed transition functions to `CechCohomologyComputer`
- Build $C^0 \xrightarrow{d^0} C^1 \xrightarrow{d^1} C^2$
- Compute $H^1 = \ker(d^1)/\operatorname{im}(d^0)$

Step 5 — Verdict:

- $H^1 = 0$: Global equivalence. Proof term assembled from local proofs.
- $H^1 \neq 0$: Not equivalent. Cocycle representatives localize the failure to specific overlaps with concrete counterexamples.

What you tell the audience:

> *"$H^1 = 0$ is sound AND complete. Standard modular checking is only sound. We never reject equivalent programs."*

Global bisimulation:

$$\text{State space} = d^k, \quad \text{Cost} = \mathcal{O}(d^k \log d^k)$$

Sheaf decomposition into m sites, each touching k/m variables:

$$\underbrace{\mathcal{O}(m \cdot d^{k/m})}_{\text{local equiv}} + \underbrace{\mathcal{O}(|\text{overlaps}| \cdot \text{SMT})}_{\text{gluing}} + \underbrace{\mathcal{O}(|\text{triples}|)}_{\text{cocycle}}$$

Example: $k = 20$ variables, $d = 100$, $m = 5$ sites.

- Global: $100^{20} = 10^{40}$ — intractable
- Sheaf: $5 \cdot 100^4 = 5 \times 10^8$ — tractable

When does this work? When variables partition into nearly-independent groups. This is the *common case*: functions have local variables.

Input: Function f + refinement-type spec.

Standard approach: One monolithic VC. n paths, k conjuncts $\rightarrow$ formula of size $\mathcal{O}(nk)$.

Deppy's product cover:

- Cover $= \{(\text{path}_i, \text{conj}_j) \mid i = 1 \ldots n, j = 1 \ldots k\}$
- Each site: one path $\times$ one conjunct $\rightarrow$ small VC
- Overlaps: paths sharing prefixes, conjuncts sharing variables

Payoffs:

1. **Parallel**: nk independent VCs
2. **Incremental**: change path 3 $\rightarrow$ re-check row 3 only
3. **Localized**: "conjunct 2 fails on path 4"
4. **Proof transfer**: shared prefixes $\Rightarrow$ skip redundant VCs

Standard: "All VCs passed $\Rightarrow$ spec holds." But how do you know VCs covered everything?

Deppy adds two checkable certificates:

1. $H^0 = 1$ (cover connected):
   - The product cover spans all paths and all conjuncts
   - No dead zones—nothing was silently skipped
2. $H^1 = 0$ (proofs consistent):
   - Local proofs on overlapping paths agree
   - No invisible interactions between conjuncts

Together: Coverage (H^0) + Consistency (H^1) + Correctness (VCs) = structured soundness argument.

No standard refinement checker provides this meta-level certificate.

Scenario: Function f with branches A (lines 1-20) and B (lines 21-40), overlap at lines 18-22.

Analysis results:

- $H^1(A) = \mathbb{F}_2$ (one bug in branch A)
- $H^1(B) = 0$ (branch B is clean)
- $H^1(A \cap B) = 0$ (overlap is clean)
- $\mathrm{im}(\delta) = \mathbb{F}_2$ (the overlap creates one additional obstruction)

Mayer-Vietoris gives:

$$\mathrm{rk}\, H^1(f) = 1 + 0 - 0 + 1 = 2$$

Developer action: Fix the bug in A ($\mathrm{rk}\, H^1(A) \to 0$), recheck only δ. If $\mathrm{im}(\delta) \to 0$ too: $\mathrm{rk}\, H^1(f) = 0$. Done.

No re-analysis of B at all.

Edit a function: Change one basic block.

Standard analyzer: Re-analyze the entire function (re-compute global fixpoint).

Deppy:

1. Identify which site(s) changed
2. Re-infer local sections at changed sites only
3. Re-check gluing at overlaps involving changed sites
4. Recompute H^1 (fast: one row of the coboundary matrix changed)

Cost: $\mathcal{O}(\text{degree of changed site})$ instead of $\mathcal{O}(n)$.

This is the "edit-check-fix" loop that makes the sheaf approach practical for IDE integration.

1. **Parse** Python $\to$ IR (core terms)
2. **Cover** from CFG: sites $U_1, \ldots, U_n$ + overlaps
3. **Local inference**: for each U_i, compute `LocalSection`(U_i)
4. **Forward/backward propagation**: refine sections until fixpoint
5. **Gluing check**: for each overlap (U_i, U_j):
   - Restrict both sections to overlap
   - Compare via Z3: $\text{gap} = \text{avail}_i \wedge \neg\, \text{req}_j$
   - SAT $\to$ record obstruction
6. H^1 **computation**: coboundary matrix $\to$ Gaussian elim over $\mathbb{F}_2$
7. **Obstruction basis**: minimal support, gap predicates, repair guards
8. **Mayer-Vietoris**: compositional bug counting
9. **Render**: diagnostics, SARIF export, severity ranking

Capability	Status	Code
14 site families	✓	`_protocols.py`
Cover builder from CFG	✓	`cover_builder.py`
Semantic presheaf	✓	`presheaf.py`
Section gluing check	✓	`section_gluing.py`
H^1 via $\mathbb{F}_2$ Gaussian elim	✓	`obstruction_extractor.py`
Gap predicate $\to$ repair guard	✓	`repair_guidance.py`
Čech cocycle on triples	✓	`cohomology.py`
$H^1 = 0 \Leftrightarrow$ global equiv	✓	`descent.py`
Product-cover VC factoring	✓	Pipeline
$H^0 + H^1$ meta-certificate	✓	Pipeline
Mayer-Vietoris δ	✓	New
Formal soundness proof (Lean)	×	Open
Benchmark vs Astrée	×	Open

Being honest:

- **Not a silver bullet**: Some bugs require global reasoning that can't be decomposed
- **Not free**: Cover construction has overhead; trivial programs don't benefit
- **Not always better than AI**: For simple interval analysis, Astrée is faster
- **Not formally verified yet**: The Lean proof is open work
- **Not benchmarked against all competitors**: Head-to-head with Liquid Haskell, F* is future work

The honest claim: The sheaf framing provides *specific, measurable advantages* (obstruction counting, completeness, compositionality) that no standard tool offers. But it's not universally superior.

The sheaf approach dominates when:

1. **Many variables, natural decomposition**:
   Variables partition into groups $\to$ exponential savings
2. **Many overlaps with simple interactions**:
   Local SMT queries are cheap; global fixpoint is expensive
3. **Incremental changes**:
   Edit one site, re-check only neighbors
4. **Equivalence checking**:
   $H^1 = 0$ completeness is unique to this approach
5. **Error triage**:
   $\mathrm{rk}\, H^1$ gives minimum fix count; standard tools just list warnings
6. **Compositional reasoning (Mayer-Vietoris)**:
   Combine analyses of parts without re-analyzing

Diminishing returns when:

1. **Tiny programs**: Cover overhead not worth it
2. **Tightly coupled variables**: No good decomposition, $m \approx 1$, no savings
3. **Simple properties**: "Is $x > 0$?" — just run interval analysis
4. **Deep recursion**: Need induction, not topology
5. **Liveness properties**: Sheaf framework is for safety (local checks), not liveness (global behavior over infinite traces)

Rule of thumb: If the standard analysis already takes < 1s and reports < 5 warnings, the sheaf machinery is overkill.

Q: "Isn't this just modular analysis with fancy vocabulary?"
A: The $H^1 = 0 \Leftrightarrow$ completeness result is new. Standard modular analysis doesn't have it.

Q: "Does the $\mathbb{F}_2$ linearization lose information?"
A: It collapses "how much" two sections disagree to "do they disagree." This is the right abstraction: we want to count bugs, not measure bug severity algebraically.

Q: "Why not just use Astrée/Infer?"
A: They don't give you obstruction counting, compositional analysis, or equivalence completeness. Different tools for different questions.

Q: "Where's the formal proof?"
A: Open work. The implementation is tested on real examples; the Lean formalization is planned.

Claim 1 (Bug finding): Given a cover $\mathcal{U}$ and local sections $\{s_i\}$, gluing failures are obstructions in $H^1(\mathcal{U}, \mathrm{Sem})$. $\mathrm{rk}\, H^1 =$ minimum independent fixes. Each obstruction carries a gap predicate whose negation is a repair guard.

Claim 2 (Equivalence): $H^1(\mathcal{U}, \mathrm{Iso}(\mathrm{Sem}_f, \mathrm{Sem}_g)) = 0$ iff local equivalences glue to a global equivalence. Sound and complete for the cover.

Claim 3 (Spec checking): Product cover factoring gives nk independent VCs. All pass $\wedge$ $H^0 = 1$ $\wedge$ $H^1 = 0 \Rightarrow$ structured soundness certificate.

Claim 4 (Compositionality): Mayer-Vietoris:
$\mathrm{rk}\, H^1(f) = \mathrm{rk}\, H^1(A) + \mathrm{rk}\, H^1(B) - \mathrm{rk}\, H^1(A \cap B) + \mathrm{rk}\ \mathrm{im}(\delta)$.

Slide 3: Checking Gluing $\neq$ Constructing Invariants

You say:

> *"Standard abstract interpretation computes a global fixpoint — that's $\mathcal{O}(|D|^k)$ per iteration, requires widening, and loses precision.*
>
> *The sheaf approach decouples this into two phases:*
>
> 1. *Local inference at each site — $\mathcal{O}(n)$, any method*
> 2. *Gluing check on overlaps — $\mathcal{O}(|overlaps|)$ local Z3 queries*
>
> *The key insight: checking whether adjacent sites agree (the restriction map is total) is fundamentally cheaper than constructing a global invariant."*

Slide 4: $\operatorname{rk} H^1$ = Minimum Independent Fixes

You say:

> *"You have 47 warnings. How many are actually independent? Standard tools don't know. We build the coboundary matrix — rows are overlaps, columns are sites, entries are 0/1 over $\mathbb{F}_2$.*
>
> *Gaussian elimination gives the rank. $\operatorname{rk} H^1 = 5$ means exactly 5 independent obstructions. Fix those 5, all 47 resolve.*
>
> *Each obstruction has a gap predicate: the formula that makes the overlap fail. Negate it $\to$ repair guard. 'Add `if y != 0:` before the division.' "*

Slide 5: Exponential Savings from Decomposition

You say:

> *"Global bisimulation for equivalence: d^k state space, exponential.*
>
> *Sheaf decomposition into m sites, each touching k/m variables: $m \cdot d^{k/m}$. That's exponentially cheaper.*
>
> *When does this work? When variables partition into nearly-independent groups — the common case. Functions have local variables. The natural cover from code structure gives $m = \Theta(k/\log k)$.*
>
> *What you pay: cocycle check on triples, $\mathcal{O}(m^3)$ worst case. But typically sparse — dominated by local equivalence checks."*

Slide 6: $H^1 = 0$ Is Sound AND Complete

You say:

> *"This is the key theoretical result. Standard modular checking: interfaces match implies equivalent. Sound but not complete — it rejects equivalent programs with non-matching interfaces.*
>
> *Our sheaf approach: $H^1 = 0$ if and only if the local equivalences glue to a global one. Sound AND complete for the cover.*
>
> *The completeness direction comes from descent theory: if the cocycle is a coboundary (i.e., $H^1 = 0$), we can adjust the local isomorphisms to be globally compatible. This is a theorem, not a heuristic."*

Slide 9: Mayer-Vietoris Compositionality

You say:

"This is the crown jewel. Decompose a function at a branch into A and B. The Mayer-Vietoris exact sequence gives:

$\mathrm{rk}\, H^1(f) = \mathrm{rk}\, H^1(A) + \mathrm{rk}\, H^1(B) - \mathrm{rk}\, H^1(A \cap B) + \mathrm{rk}\ \mathrm{im}(\delta)$

Bug count of the whole from bug counts of the parts.

Three wins: compositional bug counting, incremental re-analysis (modify A, don't touch B), and unification — the same sequence governs bugs, equivalence, and specs.

Nothing else gives you this. Not Astrée, not Infer, not Liquid Haskell."

PL

$\Gamma \vdash e : \tau$	typing judgment
$\{x : \tau \mid \varphi\}$	refinement type
Π, Σ, Id	dependent types
∇	widening
$\mathrm{wp}(c, Q)$	weakest precon.

Sheaf/Cohomology

$\mathcal{F}(U), \Gamma(U, \mathcal{F})$	sections
$\rho_{V,U}, s\vert_U$	restriction
H^0, H^1	cohomology groups
d^0, d^1	coboundary maps
∂_0	coboundary matrix
$\mathrm{rk}\, H^1$	indep. obstructions
δ	connecting hom.

Everyday	**PL**	**Sheaf**
Code region	Basic block / scope	Site U_i
Variable scope	Live variable set	Support of section
Type at a point	Typing judgment	Local section $s_i \in \mathcal{F}(U_i)$
Types agree	Subtyping holds	Gluing condition
Type error	Ill-typed expression	Obstruction in H^1
Bug count	Warning count	$\operatorname{rk} H^1$
Bug fix	Patch	Repair guard $\neg\varphi$
Same behavior	Bisimulation	$H^1(\textsc{Iso}) = 0$
Covers everything	Path coverage	$H^0 = 1$
Consistent proofs	Modular compatibility	$H^1 = 0$
Combine results	Compositional	Mayer-Vietoris

PL Foundations:

- Pierce, *Types and Programming Languages* (2002)
- Cousot & Cousot, "Abstract Interpretation" (POPL 1977)
- Rondon, Kawaguchi, Jhala, "Liquid Types" (PLDI 2008)

Sheaf Theory:

- Hartshorne, *Algebraic Geometry*, Ch. II (1977)
- Mac Lane & Moerdijk, *Sheaves in Geometry and Logic* (1994)
- Kashiwara & Schapira, *Sheaves on Manifolds* (1990)

Cohomology:

- Bott & Tu, *Differential Forms in Algebraic Topology* (1982)
- Hatcher, *Algebraic Topology*, Ch. 3 (2002) — free online
- Godement, *Topologie Algébrique et Théorie des Faisceaux* (1958)

1. What is the sheaf gluing condition, in one sentence?
2. Why is checking agreement cheaper than constructing invariants?
3. What does $\operatorname{rk} H^1 = 3$ mean for a buggy program?
4. What does $H^1 = 0$ mean for two programs f, g?
5. How does the product cover factor VCs?
6. What are the three certificates in the meta-soundness argument?
7. State the Mayer-Vietoris formula for $\operatorname{rk} H^1(f)$.
8. Why is the sheaf equivalence check complete while standard modular checking isn't?
9. What is a gap predicate? What is a repair guard?
10. Why does deppy work over $\mathbb{F}_2$?

1. Compatible local sections glue to a unique global section.
2. Restriction is total; constructing a global fixpoint requires iteration + widening.
3. 3 independent bugs; fix those 3 and all related warnings resolve.
4. Local equivalences glue to a global one: f and g are equivalent.
5. Each (path, conjunct) pair is a separate small VC.
6. $H^0 = 1$ (coverage), $H^1 = 0$ (consistency), all VCs pass (correctness).
7. $\mathrm{rk}\, H^1(f) = \mathrm{rk}\, H^1(A) + \mathrm{rk}\, H^1(B) - \mathrm{rk}\, H^1(A \cap B) + \mathrm{rk}\ \mathrm{im}(\delta)$.
8. Standard: sound only (interfaces match $\Rightarrow$ equiv). Sheaf: sound + complete ($H^1 = 0 \Leftrightarrow$ equiv). Descent theorem gives the converse.
9. Gap = avail $\wedge \neg$req. Repair = $\neg$gap.
10. Agree/disagree is binary; $\mathbb{F}_2$ is the natural coefficient field.

You're Ready.

The sheaf framing gives you three concrete tools:

$$\begin{aligned} \operatorname{rk} H^1 &= \text{minimum independent fixes} \\ H^1 = 0 &= \text{sound and complete equivalence} \\ \text{Mayer-Vietoris} &= \text{compositional obstruction counting} \end{aligned}$$

Not vocabulary. Not hand-waving.
Concrete algorithms with concrete advantages.

Go present with confidence.

Proof terms witness equivalences and type correctness:

Term	Meaning
`ReflProof`	Identity: $a = a$
`SymmetryProof`	If $a = b$ then $b = a$
`TransitivityProof`	If $a = b$ and $b = c$ then $a = c$
`TransportWitness`	Carry a proof along a path
`CompositeProof`	Chain of proof steps

These are the proof terms from deppy's `witnesses.py`. They compose to form the global proof certificate when $H^1 = 0$.

	U_1	U_2	U_3	U_4	U_5
e_{12}	1	1	0	0	0
e_{13}	1	0	1	0	0
e_{24}	0	1	0	1	0
e_{35}	0	0	1	0	1
e_{45}	0	0	0	1	1
e_{15}	1	0	0	0	1

5 sites, 6 overlaps. Rank via Gaussian elimination over $\mathbb{F}_2$: $\mathrm{rk}(\partial_0) = 4$.
$\mathrm{rk}\, H^1 = 6 - 4 = 2$ (if no triple overlaps contribute). Two independent obstructions.

`cover_builder.py` constructs the Grothendieck cover from the CFG:

1. **Node traversal**: Each CFG node $\to$ candidate site
2. **Site classification**: Classify into 14 families by AST type
3. **Overlap detection**: Two sites overlap if they share variables (value lineage) or control flow (dominator tree)
4. **Boundary identification**: Input/output sites $\to$ boundary sections
5. **Error site marking**: Division, index, attribute access $\to$ error sites
6. **Morphism construction**: For each overlap, build a `ConcreteMorphism` with projection map

Result: A `Cover` object ready for presheaf evaluation.

SARIF (Static Analysis Results Interchange Format): Standard for reporting static analysis results.

Deppy's `sarif_exporter.py` exports:

- Each obstruction as a SARIF `result`
- Gap predicate in the `message`
- Repair guard in the `fix` suggestion
- Source location from `SiteId.source_location`
- Severity from obstruction ranking
- rk H^1 in the run-level `properties`

This integrates with VS Code, GitHub Code Scanning, and any SARIF-compatible tool.

User-facing surface syntax for specs:

- `@deppy.requires(lambda x: x > 0)` — precondition
- `@deppy.ensures(lambda result: result >= 0)` — postcondition
- `@deppy.invariant(lambda self: self.count >= 0)` — class invariant
- `@deppy.decreases(lambda n: n)` — termination measure
- `@deppy.theorem` — user-supplied proof
- `@deppy.witness` — constructive proof term

Each decorator is lowered to a `BoundarySection` by the contract lowering pipeline. The sheaf machinery then checks if the function body satisfies the boundary constraints.

The deep analogy:

Geometry & **Programming**
Manifold & Program
Open cover & Code regions (sites)
Structure sheaf & Type presheaf
Global section & Consistent global typing
Tangent bundle & Variable dependencies
Cohomology class & Bug / obstruction
Euler characteristic & Program health metric
Mayer-Vietoris & Compositional analysis
Vector bundle & Equivalence class
Descent & Modular verification

“The sheaf condition is not a metaphor.
It is the algorithm.”

Check gluing. Compute H^1. Count obstructions.
Fix the minimum. Verify compositionality.
That’s it. That’s the whole thing.